Studio Nx

S1-09 악기·이펙트 API — 함수 설명 합본 (백엔드 226~242)

2026-10-18

목차

1. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/226_models_plugin.md
2. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/227_models_track_plugin.md
3. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/228_models_track_phase2.md
4. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/229_schemas_plugin.md
5. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/230_schemas_device.md
6. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/231_schemas_track.md
7. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/232_plugin_service.md
8. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/233_track_service_track.md
9. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/234_track_service_instrument.md
10. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/235_track_service_effect.md
11. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/236_track_service_link.md
12. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/237_api_plugins.md
13. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/238_api_tracks.md
14. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/239_api_instruments.md
15. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/240_api_effects.md
16. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/241_api_links.md
17. ai-notes/backend(fastAPI_Python)/S1-09 - 기본 악기·이펙트/242_seed_plugins.md

Backend(FastAPI/Python) 226 — `Plugin` · `PluginPreset`

항목	내용
파일	`studionx/backend/app/models/plugin.py`
작업	S1-09 기본 악기·이펙트: `plugins` · `plugin_presets` 표 (PRD 02 FR-241, FR-244, FR-246)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	225 `setClipNotes` · 227 `TrackInstrument` · `TrackPlugin`

1. 하는 일

가상악기와 이펙트의 **정의**를 담는 표 두 개입니다. 플러그인은 앱에 코드로 박혀 있고, 표에는 그 플러그인을 화면에 어떻게 그릴지 알려 주는 정보만 들어 있습니다.

넣은 것	뜻
`code`	화면(JS)이 구현을 찾는 열쇠. `eq3` 이면 화면의 EQ 구현과 이어집니다 (FR-241·2187)
`name`	사람에게 보이는 이름 (`Subtractive Synth`, `EQ 3-Band` …)
`plugin_type`	`INSTRUMENT`(가상악기) · `EFFECT`(이펙트) · `MIDI_EFFECT`(MIDI 이펙트)
`engine`	소리를 만드는 방식. `WEBAUDIO`(Web Audio 노드 조합) · `WASM`(AudioWorklet, FR-247)
`param_schema`	파라미터 목록(JSON). 이름, 범위, 기본값, 단위, `group`(모달의 칸), `primary`(카드에 노출할 핵심값)가 들어 있고, 카드와 상세 모달이 이 값을 읽어 노브를 자동으로 만듭니다 (FR-244)
`card_asset`	Nyxi 카드 그림 파일 이름 (VARCHAR(100)). 그림은 외부 AI 플랫폼에서 받으므로 지금은 비어 있습니다 (FR-2181)
`is_active`	지금 쓸 수 있는 플러그인인지 (꺼진 것은 목록에서 빠집니다)

`plugin_presets` 는 그 플러그인 위에 붙는 **저장된 파라미터 묶음**입니다. `owner_id` 가 비어 있으면 모두에게 보이는 시스템 프리셋, 값이 있으면 그 사용자가 저장한 개인 프리셋입니다 (FR-246).

넣은 것	뜻
`plugin_id`	어느 플러그인의 프리셋인지 (FK → `plugins`)
`owner_id`	주인 (FK → `users`). 비어 있으면 시스템 기본 프리셋
`name`	프리셋 이름 (`Init`, `Mine` …)
`params`	그 프리셋의 파라미터 값 묶음 (JSON)
`source`	`SYSTEM` · `USER` · `AI`. AI 가 추천해 저장한 것은 `AI` 입니다 (DB 기본값은 `USER`)

2. 언제 쓰나

쓰는 곳	
`plugin_service.list_plugins` · `require_plugin` (232)	플러그인 목록과 한 건을 읽습니다
`plugin_service.list_plugin_presets` (232)	시스템 프리셋과 내 프리셋만 골라 읽습니다

쓰는 곳	
`track_service.add_instrument` · `add_effect` (234·235)	붙이려는 플러그인의 종류를 확인합니다
시드 SQL (242)	기본 악기 2종과 이펙트 5종을 이 표에 넣습니다

3. 코드

종류·방식·출처 (열거형)

```
class PluginType(StrEnum):
    INSTRUMENT = "INSTRUMENT"
    EFFECT = "EFFECT"
    MIDI_EFFECT = "MIDI_EFFECT"


class PluginEngine(StrEnum):
    WEBAUDIO = "WEBAUDIO"
    WASM = "WASM"


class PresetSource(StrEnum):
    SYSTEM = "SYSTEM"
    USER = "USER"
    AI = "AI"
```

`Plugin`

```
class Plugin(Base):
    __tablename__ = "plugins"

    plugin_id: Mapped[int] = mapped_column(INTEGER(unsigned=True), primary_key=True, autoincrement=True)
    code: Mapped[str] = mapped_column(String(50))
    name: Mapped[str] = mapped_column(String(100))
    plugin_type: Mapped[PluginType] = mapped_column(Enum(PluginType))
    engine: Mapped[PluginEngine] = mapped_column(Enum(PluginEngine), server_default=PluginEngine.WEBAUDI
O)
    version: Mapped[str] = mapped_column(String(20), server_default=text("'1.0.0'"))
    param_schema: Mapped[list[dict[str, Any]]] = mapped_column(JSON)
    card_asset: Mapped[str | None] = mapped_column(String(100))
    is_active: Mapped[bool] = mapped_column(Boolean, server_default=text("1"))
```

`PluginPreset` (핵심 칸)

```
    plugin_id: Mapped[int] = mapped_column(ForeignKey("plugins.plugin_id"))
    owner_id: Mapped[int | None] = mapped_column(ForeignKey("users.user_id"))
    name: Mapped[str] = mapped_column(String(100))
    params: Mapped[dict[str, Any]] = mapped_column(JSON)
    source: Mapped[PresetSource] = mapped_column(Enum(PresetSource), server_default=PresetSource.USER)
```

문법 풀이

식	뜻
`StrEnum`	문자열이면서 열거형입니다. `PluginType.INSTRUMENT == "INSTRUMENT"` 가 참이라 DB·JSON 과 그대로 주고받습니다
`Enum(PluginType)`	DB 에는 `ENUM('INSTRUMENT','EFFECT','MIDI_EFFECT')` 로 만들어집니다 (DB.md 와 같습니다)
`server_default=PresetSource.USER`	값을 안 주면 DB 가 `'USER'` 로 채웁니다. DB.md 의 기본값과 같게 맞췄습니다
`Mapped[list[dict[str, Any]]]`	JSON 배열임을 타입으로 알려 줍니다. 실제 값은 `param_schema` 항목의 목록입니다
`owner_id: Mapped[int | None]`	비어 있을 수 있음(NULL)을 뜻합니다. 시스템 프리셋이 이 경우입니다
`passive_deletes=True`	프리셋 삭제를 DB 의 `ON DELETE CASCADE` 에 맡기고 앱이 하나씩 지우지 않습니다

4. 왜 이렇게

선택	이유
`code` 로 화면 구현을 찾음	플러그인마다 화면 코드를 따로 만들면 늘어날수록 관리가 어렵습니다. `code` 하나로 표와 화면을 잇습니다 (FR-241·2187)
그리는 정보를 `param_schema` 에 모음	카드와 상세 모달이 같은 값을 읽어 자동으로 만들어집니다. 플러그인을 추가해도 화면 코드를 새로 쓰지 않습니다 (FR-244·2187)
`owner_id` 로 시스템·개인을 구분	표를 나누지 않고 한 표에서 `owner_id IS NULL`(시스템) 과 `owner_id = 나`(개인) 만 골라 읽습니다
`card_asset` 을 비워 둠	캐릭터 그림은 외부 AI 플랫폼에서 만듭니다. 프로젝트 원칙에 따라 AI 가 그림을 만들지 않습니다 (FR-2181)
`engine` 을 남겨 둠	지금은 Web Audio 로 만들지만, 나중에 WASM(AudioWorklet) 플러그인을 같은 표에 담기 위해서입니다 (FR-247)

5. 시험

`tests/test_plugin_model.py` 에서 두 표의 칸을 실제 MySQL 과 비교(`assert_matches_db`)하고, DB 기본값(`engine` = `WEBAUDIO`, `is_active` = 1, `source` = `USER`), `param_schema` JSON 왕복, 프리셋 `owner_id` 구분을 확인합니다.

> `plugins.code` 는 DB 에서 UNIQUE 입니다. 시드를 두 번 실행해도 `ON DUPLICATE KEY UPDATE` 로 갱신만 됩니다 (242).

Backend(FastAPI/Python) 227 — `TrackInstrument` · `TrackPlugin` · `TrackPluginLink`

항목	내용
파일	`studionx/backend/app/models/track_plugin.py`
작업	S1-09 기본 악기·이펙트: 트랙 디바이스 표 세 개 (PRD 02 FR-245, FR-263, FR-267, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	226 `Plugin` · 228 `Track` Phase 2 칸

1. 하는 일

트랙 한 줄의 **소리 만들기 구성**을 담는 표 세 개입니다. 정의(226 `plugins`)와 실제로 트랙에 붙은 칸을 나눠 두었습니다.

표	넣은 것	뜻
`track_instruments`	`track_id`, `plugin_id`, `slot_index`(기본 0), `params`, `volume_db`, `pan`, `is_solo`, `is_enabled`	트랙의 악기 한 칸. `params` 에 그 악기의 파라미터가 들어갑니다 (FR-245·264)
`track_plugins`	위와 같고 `volume_db`·`pan`·`is_solo` 대신 `node_x`, `node_y`, `is_bypassed`	트랙의 이펙트 한 칸. `slot_index` 가 체인 순서, `node_x`·`node_y` 는 갈래 화면의 카드 자리입니다 (FR-263·269)
`track_plugin_links`	`track_id`, `from_track_plugin_id`, `from_port`, `to_track_plugin_id`, `to_port`, `gain_db`	갈래를 나누고 합치는 연결선입니다. `from` 하나에서 여럿으로 가면 갈래, 여럿이 `to` 하나로 모이면 병합입니다 (FR-267)

`from_track_plugin_id` 나 `to_track_plugin_id` 가 비어 있으면(NULL) 각각 트랙 입력(소스)과 트랙 출력을 뜻합니다.

2. 언제 쓰나

쓰는 곳	
`track_service.list_instruments` · `add_instrument` · `update_instrument` · `remove_instrument` (234)	악기 칸을 읽고 붙이고 고치고 뗍니다
`track_service.list_effects` · `add_effect` · `update_effect` · `remove_effect` (235)	이펙트 칸을 다룹니다
`track_service.list_links` · `add_link` · `remove_link` (236)	연결선을 다룹니다
`Track.instruments` · `Track.plugins` · `Track.plugin_links` 관계 (228)	트랙을 지울 때 DB CASCADE 로 함께 지워집니다

3. 코드

`TrackInstrument`

```
    track_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE"))
    plugin_id: Mapped[int] = mapped_column(INTEGER(unsigned=True), ForeignKey("plugins.plugin_id"))
    slot_index: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("0"))
    params: Mapped[dict[str, Any] | None] = mapped_column(JSON)
    volume_db: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
    pan: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    is_solo: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    is_enabled: Mapped[bool] = mapped_column(Boolean, server_default=text("1"))
```

`TrackPlugin`

```
    slot_index: Mapped[int] = mapped_column(TINYINT(unsigned=True))          # 체인 순서 (필수)
    node_x: Mapped[int | None] = mapped_column(INTEGER)                      # 갈래 화면 자리
    node_y: Mapped[int | None] = mapped_column(INTEGER)
    params: Mapped[dict[str, Any]] = mapped_column(JSON)
    is_bypassed: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
```

`TrackPluginLink`

```
    track_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE"))
    from_track_plugin_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("track_plugins.track_plugin_id", ondelete="CASCADE")
    )
    from_port: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("0"))
    to_track_plugin_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("track_plugins.track_plugin_id", ondelete="CASCADE")
    )
    to_port: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("0"))
    gain_db: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
```

문법 풀이

식	뜻
`ondelete="CASCADE"`	부모(트랙, 이펙트)를 지우면 이 행도 DB 가 함께 지웁니다
`ForeignKey("plugins.plugin_id")` (ON DELETE 없음)	쓰이던 플러그인은 지울 수 없게 DB 가 막습니다. 실수로 소리가 사라지는 일을 막습니다
`TINYINT(unsigned=True)`	0~255 정수입니다. 칸 번호와 포트 번호는 음수가 될 수 없습니다
`Mapped[int | None]`	비어 있을 수 있는 칸입니다. `node_x`·`from`·`to` 가 그렇습니다
`server_default=text("0")`	값을 안 주면 DB 가 채웁니다. 응답에는 실제 값이 담겨 나옵니다
`JSON`	파라미터 묶음을 그대로 담습니다. 플러그인마다 파라미터가 달라도 표를 바꾸지 않습니다

4. 왜 이렇게

선택	이유
악기와 이펙트를 다른 표로	악기는 트랙에 하나 이상 겹쳐 놓고(레이어) 볼륨·팬·솔로를 따로 가집니다. 이펙트는 체인 순서와 갈래 자리를 가집니다. 담는 칸이 달라 표를 나눴습니다
빈 칸 대신 `NULL` 로 시작·끝	연결선을 한 표로 다루면서 트랙 입력과 출력을 함께 그릴 수 있습니다 (FR-267·269)

선택	이유
연결선을 따로 저장	갈래는 한 이펙트에서 여럿으로 나가고 다시 합쳐집니다. 나무 구조는 표 한 줄로 담을 수 없습니다 (FR-265·267)
칸 번호에 UNIQUE 를 둠	같은 트랙의 같은 칸에 두 행이 들어가면 화면 순서를 정할 수 없습니다. DB 가 `UNIQUE (track_id, slot_index)` 로 막고, 서비스가 먼저 확인해 400 으로 알려 줍니다 (234 §3)
`passive_deletes=True`	삭제는 DB CASCADE 에 맡깁니다. 앱이 자식 행을 하나씩 지우다 빠뜨릴 일이 없습니다

5. 시험

`tests/test_plugin_model.py` 에서 세 표의 칸을 실제 MySQL 과 비교하고, 같은 칸에 두 번 넣으면 `IntegrityError` 가 나는지, 트랙을 지우면 세 표의 행이 모두 사라지는지 확인합니다.

> 트랙을 지운 뒤 앱이 들고 있던 관계 목록(`track.instruments`)에는 지워진 행이 남을 수 있습니다. 그래서 목록 API 는 관계 대신 쿼리로 다시 읽습니다 (234 참고).

Backend(FastAPI/Python) 228 — `Track` Phase 2 칸 · `RoutingMode`

항목	내용
파일	`studionx/backend/app/models/track.py`
작업	S1-09 기본 악기·이펙트: 트랙 헤더 믹서와 라우팅 방식 칸 (PRD 02 FR-251, FR-252, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	227 `TrackInstrument` · `TrackPlugin` · 229 플러그인 스키마

1. 하는 일

`tracks` 표에 Phase 2 에서 쓰는 칸을 모두 채웠습니다. Phase 1 에서는 프로젝트를 만들 때 MASTER 트랙 한 줄을 넣는 데만 썼는데(PRD 01 FR-122), 이제 트랙 헤더의 믹서(볼륨·팬·뮤트·솔로), 라우팅 방식, 갈래 자리까지 다룹니다.

새로 쓴 칸	뜻
`color`	트랙 색 `#RRGGBB`, 기본 `#7C5CFF`
`volume_db`, `pan`	페이더(-60.00~+6.00 dB)와 좌우 위치(-1.00~1.00), DECIMAL(5,2)·(3,2)
`is_muted`, `is_solo`, `is_enabled`	소리 끄기, 솔로, 트랙 자체를 멈춤. `is_enabled` 가 0이면 처리를 아예 쉽니다
`parent_track_id`	그룹 트랙에 속한 트랙 (그룹을 지우면 NULL, FR-252)
`frozen_sample_id`	프리즈 결과 오디오. 값이 있으면 악기·이펙트 처리를 쉬고 이 오디오를 재생합니다
`routing_mode`	`CHAIN`(일렬 체인) 또는 `PATCHER`(갈래 구성), 기본 `CHAIN` (FR-267)
`assignee_id`, `locked_by`, `locked_at`	담당 멤버와 잠금 정보 (Phase 6 용, 지금은 응답에만 담깁니다)

`RoutingMode` 는 이 표에서만 쓰는 열거형입니다. 화면은 이 값을 보고 [Effects] 칸을 체인으로 그릴지 갈래 카드로 그릴지 정합니다 (FR-262·267).

2. 언제 쓰나

쓰는 곳	
`track_service.list_tracks` · `create_track` · `require_track` · `update_track` (233)	트랙 한 줄을 읽고 만들고 고칩니다
`track_service.add_effect` (235)	이펙트를 붙일 때 `node_x`·`node_y`(갈래 자리)를 함께 담습니다
`GET /tracks/{id}/...` 응답 (238~241)	`TrackResponse` 가 이 칸들을 그대로 내보냅니다
S1-10 저장·불러오기	트랙과 라우팅 방식을 함께 저장하고 되살립니다 (FR-254)

3. 코드

열거형

```
class TrackType(StrEnum):
    AUDIO = "AUDIO"
    MIDI = "MIDI"
    MASTER = "MASTER"
    GROUP = "GROUP"
    RETURN = "RETURN"


class RoutingMode(StrEnum):
    """이펙트 배치 방식. CHAIN 은 한 줄(직렬), PATCHER 는 갈래(병렬)가 있는 구성 (FR-267)."""

    CHAIN = "CHAIN"
    PATCHER = "PATCHER"
```

칸

```
    order_index: Mapped[int] = mapped_column(Integer, server_default=text("0"))
    color: Mapped[str] = mapped_column(String(7), server_default=text("'#7C5CFF'"))
    volume_db: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
    pan: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    is_muted: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    is_solo: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    is_enabled: Mapped[bool] = mapped_column(Boolean, server_default=text("1"))
    parent_track_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("tracks.track_id", ondelete="SET NULL")
    )
    frozen_sample_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("samples.sample_id", ondelete="SET NULL")
    )
    routing_mode: Mapped[RoutingMode] = mapped_column(Enum(RoutingMode), server_default=RoutingMode.CHAI
N)
```

자식 관계 (S1-09 에서 추가)

```
    instruments: Mapped[list["TrackInstrument"]] = relationship(
        back_populates="track",
        order_by="TrackInstrument.slot_index",
        cascade="all, delete-orphan",
        passive_deletes=True,
    )
    plugins: Mapped[list["TrackPlugin"]] = relationship(
        back_populates="track", order_by="TrackPlugin.slot_index",
        cascade="all, delete-orphan", passive_deletes=True,
    )
    plugin_links: Mapped[list["TrackPluginLink"]] = relationship(
        back_populates="track", cascade="all, delete-orphan", passive_deletes=True
    )
```

문법 풀이

식	뜻
`server_default=text("'#7C5CFF'")`	작은따옴표가 두 번 겹친 이유는, 바깥따옴표 안의 `#7C5CFF` 를 SQL 문자열로 넘기기 위해서입니다

식	뜻
`Numeric(5, 2)`	소수 둘째 자리까지입니다. 볼륨 -60.00~+6.00 dB 를 담습니다
`ondelete="SET NULL"`	부모가 지워지면 이 칸만 비웁니다. 그룹이 사라져도 트랙은 남습니다 (FR-252)
`parent_track_id` 가 자기 표를 가리킴	같은 표를 다시 가리키는 self 참조입니다. 그룹과 그 안의 트랙을 같은 표에서 다룹니다
`order_by="TrackInstrument.slot_index"`	관계로 읽을 때 칸 번호 순으로 정렬합니다. 문자열 이름이라 모델 사이 순환을 피합니다
`cascade="all, delete-orphan"` + `passive_deletes=True`	앱은 자식을 기억만 하고, 실제 삭제는 DB CASCADE 가 합니다

4. 왜 이렇게

선택	이유
`routing_mode` 를 트랙에 둠	갈래(패처) 구성은 트랙 단위로 정해집니다. 이펙트마다 정하면 한 트랙 안에서 두 방식이 섞입니다 (FR-267)
`is_enabled` 와 `is_muted` 를 나눔	`is_muted` 는 소리만 끄고 처리는 계속합니다. `is_enabled` 는 처리를 아예 멈춥니다 (DB.md 설명과 같습니다)
`assignee_id`·`locked_by` 를 지금 채움	Phase 6 협업 칸이지만 DB.md 에 이미 있는 칸입니다. 응답 모양을 나중에 바꾸지 않으려고 지금 채웠습니다
MASTER 트랙을 별도 표가 아니라 `track_type` 으로	마스터도 트랙과 똑같이 볼륨·이펙트·라우팅을 가집니다. 표를 나누면 같은 기능을 두 번 만들어야 합니다 (FR-252)

5. 시험

`tests/test_track_model.py` 가 `tracks` 칸을 실제 MySQL 과 비교하고, Phase 2 칸의 DB 기본값(색 `#7C5CFF`, 볼륨 0.00, `routing_mode` = `CHAIN`)과 프로젝트를 지우면 트랙이 함께 지워지는지 확인합니다.
`tests/test_track_routes.py` 는 만들기·수정 응답의 기본값을 확인합니다.

Backend(FastAPI/Python) 229 — 플러그인 스키마 (`ParamSchemaItem`)

항목	내용
파일	`studionx/backend/app/schemas/plugin.py`
작업	S1-09 기본 악기·이펙트: `param_schema` 항목과 플러그인 응답 (PRD 02 FR-241, FR-244, FR-246, FR-2187)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	228 `Track` Phase 2 · 230 디바이스 스키마

1. 하는 일

플러그인 API 가 주고받는 모양을 정합니다. 가장 중요한 것은 `ParamSchemaItem` 입니다. DB 의 `plugins.param_schema`(JSON) 안에 들어 있는 파라미터 한 개를 코드로 표현한 것으로, 화면이 이 값을 읽어 노브·슬라이더·선택 상자를 **자동으로 만듭니다** (FR-244·2187).

모델	하는 일
`ParamOption`	선택 상자(`select`)의 한 항목. `value`(코드가 쓰는 값)와 `label`(사람이 보는 이름)
`ParamSchemaItem`	파라미터 한 개. 이름, 종류, 범위, 기본값, 단위, 묶음(`group`), 핵심 여부(`primary`), 로그 곡선(`curve`)
`PluginResponse`	플러그인 한 건. `code`, `name`, `plugin_type`, `engine`, `version`, `param_schema`, `card_asset`, `is_active`
`PluginListResponse`	목록. `items` 만 담습니다 (총 개수는 플러그인이 많지 않아 넣지 않았습니다)
`PluginPresetResponse`	프리셋 한 건. `owner_id` 가 비어 있으면 시스템 프리셋입니다

2. 언제 쓰나

쓰는 곳	
`GET /plugins` · `GET /plugins/{id}` (237)	목록과 상세 응답을 이 모양으로 내보냅니다
`GET /plugins/{id}/presets` (237)	`PluginPresetResponse` 목록을 내보냅니다
화면의 카드·상세 모달	`primary` 인 항목 2~4개를 카드에, `group` 별로 나머지를 모달에 그립니다 (FR-2181·2185)
시드 SQL (242)	이 구조와 같은 JSON 을 `param_schema` 에 넣습니다

3. 코드

파라미터 한 개

```
class ParamSchemaItem(BaseModel):
    key: str
    name: str
```

```
    type: Literal["knob", "select", "toggle"] = "knob"
    group: str | None = None
    min: float | int | None = None
    max: float | int | None = None
    default: float | int | bool | str | None = None
    unit: str | None = None
    step: float | int | None = None
    curve: Literal["linear", "log"] | None = None
    primary: bool | None = None
    options: list[ParamOption] | None = None
```

플러그인 응답

```
class PluginResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    plugin_id: int
    code: str
    name: str
    plugin_type: PluginType
    engine: PluginEngine
    version: str
    param_schema: list[ParamSchemaItem]
    card_asset: str | None
    is_active: bool
```

문법 풀이

식	뜻
`Literal["knob", "select", "toggle"]`	셋 중 하나만 허용한다는 뜻입니다. 오타가 있으면 요청·응답 단계에서 걸립니다
`float | int | None`	소수, 정수, 없음 셋 다 됩니다. `cutoff` 는 소수, `pitch` 는 정수를 쓰기 때문입니다
`group: str | None = None`	값이 없어도 되는 칸입니다. 선택 항목은 기본값을 함께 적어 두면 빼도 됩니다
`ConfigDict(from_attributes=True)`	ORM 객체를 그대로 넣어도 읽어 준다는 설정입니다 (`PluginResponse.model_validate(plugin)`)
`default: ... | bool | str`	켜기/끄기와 선택 항목의 기본값까지 담으려고 넓혔습니다

4. 왜 이렇게

선택	이유
한 항목을 한 모델로	화면이 파라미터마다 따로 코드를 쓰지 않고, 이 구조를 돌면서 그립니다 (FR-244·2187)
`primary` 를 둠	카드에는 핵심 2~4개만 보여 줍니다. 어느 것이 핵심인지는 플러그인이 정합니다 (FR-2181)
`group` 을 둠	상세 모달에서 Osc 1, Filter 처럼 칸을 나눕니다 (FR-2185)
`curve` 를 둠	소리는 사람이 로그 눈금으로 듣습니다. `curve: "log"` 인 값(주파수)은 화면이 로그 노브로 그립니다
`type` 기본값을 `knob` 으로	대부분이 노브입니다. 시드의 47개 항목 중 41개가 종류를 적지 않아도 됩니다 (선택 4개, 켜기/끄기 2개만 적었습니다)
`Literal` 로 종류를 제한	`plugins.plugin_type` 등이 DB ENUM 과 같아야 합니다. 화면이 대소문자로 헷갈리지 않습니다

5. 시험

`tests/test_plugin_routes.py` 가 목록·상세·프리셋 응답의 칸 이름과 값(`code`, `param_schema` 항목의 `primary`, `owner_id` 구분)을 확인합니다. 시험은 시드와 겹치지 않도록 고유한 `code`(`pl_c_eq` 등)를 씁니다.

Backend(FastAPI/Python) 230 — 디바이스 스키마 (악기 · 이펙트 · 연결선)

항목	내용
파일	`studionx/backend/app/schemas/device.py`
작업	S1-09 기본 악기·이펙트: 악기·이펙트·연결선 요청과 응답 (PRD 02 FR-245, FR-263, FR-267, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	229 플러그인 스키마 · 231 트랙 스키마

1. 하는 일

트랙에 붙는 악기(`track_instruments`), 이펙트(`track_plugins`), 연결선(`track_plugin_links`)의 요청과 응답 모양을 정합니다. 요청 6개, 응답 3개를 둡니다.

모델	하는 일
`TrackInstrumentCreateRequest`	악기 붙이기. `plugin_id` 필수, `slot_index`, `params`, `volume_db`, `pan`, `is_solo`, `is_enabled`
`TrackInstrumentUpdateRequest`	악기 고치기. 모든 칸이 선택입니다 (보낸 칸만 바뀝니다)
`TrackInstrumentResponse`	악기 한 칸의 응답
`TrackPluginCreateRequest`	이펙트 붙이기. `slot_index`(체인 순서), `node_x`, `node_y`, `params`, `is_bypassed`
`TrackPluginUpdateRequest`	이펙트 고치기 (순서 바꾸기, 카드 자리 옮기기, 파라미터, 바이패스)
`TrackPluginResponse`	이펙트 한 칸의 응답
`TrackPluginLinkCreateRequest`	연결선 만들기. `from`, `to`, 포트, `gain_db`
`TrackPluginLinkUpdateRequest`	연결선 고치기. 지금 API 에는 연결선 수정 경로가 없어 보내지 않습니다 (남겨 둔 여지만 씁니다)
`TrackPluginLinkResponse`	연결선 한 줄의 응답

주요 범위는 DB.md 를 그대로 따릅니다. 볼륨은 -60.00~+24.00 dB, 팬은 -1.00~1.00, 칸 번호와 포트는 0 이상입니다.

2. 언제 쓰나

쓰는 곳	
`POST /tracks/{id}/instruments` · `PATCH /tracks/{id}/instruments/{iid}` (239)	악기 붙이기·고치기 요청을 받습니다
`POST /tracks/{id}/plugins` · `PATCH /tracks/{id}/plugins/{pid}` (240)	이펙트 붙이기·고치기 요청을 받습니다
`POST /tracks/{id}/links` (241)	연결선 만들기 요청을 받습니다
`track_service` (233~236)	요청 값을 함수 인자로 넘기고 응답을 만듭니다

3. 코드

악기 요청 (붙이기 / 고치기)

```
class TrackInstrumentCreateRequest(BaseModel):
    plugin_id: int = Field(ge=1)
    slot_index: int = Field(ge=0, le=127, default=0)
    params: dict[str, Any] | None = None
    volume_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
    pan: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    is_solo: bool = False
    is_enabled: bool = True


class TrackInstrumentUpdateRequest(BaseModel):
    slot_index: int | None = Field(default=None, ge=0, le=127)
    params: dict[str, Any] | None = None
    volume_db: Decimal | None = Field(default=None, ge=-60, le=24)
    pan: Decimal | None = Field(default=None, ge=-1, le=1)
    is_solo: bool | None = None
    is_enabled: bool | None = None
```

이펙트 요청 (붙이기)

```
class TrackPluginCreateRequest(BaseModel):
    plugin_id: int = Field(ge=1)
    slot_index: int = Field(ge=0, le=127, default=0)
    node_x: int | None = None
    node_y: int | None = None
    params: dict[str, Any] = Field(default_factory=dict)
    is_bypassed: bool = False
```

연결선 요청 (만들기)

```
class TrackPluginLinkCreateRequest(BaseModel):
    from_track_plugin_id: int | None = None
    from_port: int = Field(default=0, ge=0, le=15)
    to_track_plugin_id: int | None = None
    to_port: int = Field(default=0, ge=0, le=15)
    gain_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
```

문법 풀이

식	뜻
`Field(ge=1)`	1 이상이어야 합니다. 없는 번호(0, 음수)를 미리 막아 422 로 답합니다
`Field(default=Decimal("0.00"), ge=-60, le=24)`	기본값과 함께 아래위 한계를 정합니다. 문자열 `"0.00"` 을 `Decimal` 로 바꿔 정확한 소수를 씁니다
`Field(default_factory=dict)`	매 요청마다 **새** 빈 사전을 만듭니다. `default={}` 로 쓰면 같은 사전을 함께 쓰는 위험이 있습니다
`int | None = None` (고치기 요청)	안 보낸 칸과 0 을 보낸 칸을 구분합니다. 라우터가 `exclude_unset=True` 로 안 보낸 칸을 걸러 냅니다

식	뜻
`Literal["CHAIN", "PATCHER"]`	라우팅 방식 두 값만 허용합니다 (231)

4. 왜 이렇게

선택	이유
붙이기와 고치기를 따로	붙일 때는 값이 다 있어야 하고(기본값 포함), 고칠 때는 보낸 칸만 바뀌어야 합니다. 한 모델로 두면 빈 값이 덮어쓸 수 있습니다
범위를 스키마에서 미리 막음	서비스와 DB 까지 가지 않고 422 로 바로 답합니다. DB CHECK 와 같은 한계를 화면에도 알려줍니다
`params` 를 `dict` 로	플러그인마다 파라미터가 다릅니다. 스키마를 플러그인마다 만들지 않고 사전으로 받습니다
`node_x`·`node_y` 를 빼도 되게	체인(CHAIN) 방식에서는 자리를 쓰지 않습니다. 갈래(PATCHER)로 바꿀 때만 넣습니다 (FR-269)
연결선에 `from`·`to` 를 비울 수 있게	비어 있으면 트랙 입력과 출력입니다. 한 표로 갈래 전체를 그릴 수 있습니다 (FR-267)
`TrackPluginLinkUpdateRequest` 를 남겨 둠	지금은 선을 지우고 다시 긋지만, 나중에 선만 옮기는 화면이 생기면 이 모델을 씁니다

5. 시험

`tests/test_track_routes.py` 가 악기·이펙트·연결선의 붙이기, 고치기, 떼기와 함께 다음을 확인합니다.

- 붙이기 응답의 칸 이름(`INSTRUMENT_KEYS`, `EFFECT_KEYS`, `LINK_KEYS`)과 기본값(`is_enabled` = true, `is_bypassed` = false)
- 악기 자리에 이펙트를 넣으면 400, 이펙트 자리에 악기를 넣으면 400
- 범위를 벗어난 값(색 형식, 볼륨 99.00, 칸 번호)은 422
- 같은 칸에 두 번 넣거나 남이 쓰는 칸으로 옮기면 400

Backend(FastAPI/Python) 231 — 트랙 스키마 (만들기 · 고치기 · 응답)

항목	내용
파일	`studionx/backend/app/schemas/track.py`
작업	S1-09 기본 악기·이펙트: 트랙 요청과 응답 (PRD 02 FR-251, FR-263, FR-264)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	230 디바이스 스키마 · 232 플러그인 서비스

1. 하는 일

트랙 목록·한 건·만들기·고치기의 요청과 응답 모양을 정합니다. 트랙 헤더 믹서가 다루는 칸(색, 볼륨, 팬, 뮤트, 솔로, 켜기)과 라우팅 방식, 그룹 부모, 순서가 들어 있습니다.

모델	하는 일
`TrackBase`	만들기에서 쓰는 공통 칸. 이름은 1~100자, 색은 `#RRGGBB` 형식
`TrackCreateRequest`	`TrackBase` 에 `project_id`, `order_index`, `parent_track_id` 를 더합니다
`TrackUpdateRequest`	고치기. 모든 칸이 선택이라 보낸 칸만 바뀝니다
`TrackResponse`	응답 한 건. `Track` 모델 칸을 그대로 내보냅니다
`TrackListResponse`	목록. `items` 와 `total`

2. 언제 쓰나

쓰는 곳	
`GET /projects/{id}/tracks` · `POST` (238)	목록과 만들기 요청·응답
`GET /tracks/{id}` · `PATCH` · `DELETE` (238)	한 건 조회, 고치기, 지우기
`track_service.create_track` · `update_track` (233)	요청 값을 함수 인자로 넘깁니다
화면의 트랙 헤더	색·볼륨·팬·뮤트·솔로를 이 모양으로 보냅니다

3. 코드

공통 칸과 만들기

```
TrackType = Literal["AUDIO", "MIDI", "MASTER", "GROUP", "RETURN"]
RoutingMode = Literal["CHAIN", "PATCHER"]


class TrackBase(BaseModel):
    name: str = Field(min_length=1, max_length=100)
    track_type: TrackType = "MIDI"
    color: str = Field(default="#7C5CFF", pattern=r"^#[0-9A-Fa-f]{6}$")
    volume_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
```

```
    pan: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    is_muted: bool = False
    is_solo: bool = False
    is_enabled: bool = True
    routing_mode: RoutingMode = "CHAIN"


class TrackCreateRequest(TrackBase):
    project_id: int = Field(ge=1)
    order_index: int = Field(default=0, ge=0)
    parent_track_id: int | None = None
```

고치기 (종류 `track_type` 은 바꾸지 않습니다)

```
class TrackUpdateRequest(BaseModel):
    name: str | None = Field(default=None, min_length=1, max_length=100)
    color: str | None = Field(default=None, pattern=r"^#[0-9A-Fa-f]{6}$")
    volume_db: Decimal | None = Field(default=None, ge=-60, le=24)
    pan: Decimal | None = Field(default=None, ge=-1, le=1)
    is_muted: bool | None = None
    is_solo: bool | None = None
    is_enabled: bool | None = None
    routing_mode: RoutingMode | None = None
    order_index: int | None = Field(default=None, ge=0)
    parent_track_id: int | None = None
```

응답

```
class TrackResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    track_id: int
    project_id: int
    name: str
    track_type: TrackType
    order_index: int
    color: str
    volume_db: Decimal
    pan: Decimal
    is_muted: bool
    is_solo: bool
    is_enabled: bool
    parent_track_id: int | None
    frozen_sample_id: int | None
    routing_mode: RoutingMode
    assignee_id: int | None
    locked_by: int | None
    locked_at: str | None = None
```

문법 풀이

식	뜻
`class TrackBase(BaseModel)`	공통 칸을 한 곳에 모으고 상속으로 재사용합니다
`pattern=r"^#[0-9A-Fa-f]{6}$"`	정규식 검사입니다. `#` 뒤에 16진수 6자리만 통과합니다. `red` 는 422 입니다

식	뜻
`Literal[...]` 로 종류 제한	`AUDIO`·`MIDI`·`MASTER`·`GROUP`·`RETURN` 만 허용합니다. DB ENUM 과 같습니다
`Decimal("0.00")`	소수를 정확히 다룹니다. 부동소수(0.1+0.2) 오차가 dB 계산에 끼지 않습니다
`int | None = None` (고치기)	안 보낸 칸은 None 입니다. 라우터가 `exclude_unset=True` 로 걸러 서비스에 넘기지 않습니다
`locked_at: str | None`	값이 없을 수 있는 시각 칸입니다. 지금은 화면이 쓰지 않아 문자열로 두었습니다
`ConfigDict(from_attributes=True)`	ORM 객체를 그대로 넣어도 읽습니다

4. 왜 이렇게

선택	이유
상속으로 공통 칸 재사용	만들기와 고치기의 칸 목록이 같습니다. 두 번 쓰면 한쪽만 고치는 실수가 납니다
이름을 1자 이상으로	빈 이름 트랙은 화면에서 고를 수 없습니다. 422 로 미리 막습니다
`frozen_sample_id`·`assignee_id`·`locked_by` 까지 응답에	S1-10 저장과 Phase 6 협업에서 쓸 칸입니다. 지금 넣어 두면 화면이 응답 모양을 다시 짤 필요가 없습니다
고치기에서 `track_type` 을 뺌	트랙 종류를 바꾸면 악기·이펙트 구성이 어긋납니다. 종류는 만들 때 정하고 바꾸지 않습니다
`TrackListResponse` 에 `total` 포함	화면이 "트랙 3개" 처럼 개수를 바로 보여 줍니다 (플러그인 목록과 달리 트랙은 자주 셉니다)

5. 시험

`tests/test_track_routes.py` 가 만들기 201과 기본값(`track_type` = `MIDI`, 색 `#7C5CFF`, `routing_mode` = `CHAIN`, 볼륨 `0.00`), 고치기(이름·볼륨·라우팅 방식), 형식 오류 422, 목록 순서(`order_index`, `track_id`)를 확인합니다. `tests/test_track_model.py` 는 `tracks` 칸을 실제 MySQL 과 비교합니다.

Backend(FastAPI/Python) 232 — `plugin_service` (목록 · 한 건 · 프리셋)

항목	내용
파일	`studionx/backend/app/services/plugin_service.py`
작업	S1-09 기본 악기·이펙트: 플러그인 조회 함수 넷 (PRD 02 FR-241, FR-244, FR-246)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	231 트랙 스키마 · 233 트랙 서비스

1. 하는 일

플러그인 정의를 읽는 함수 네 개를 둡니다. 쓰기(만들기·고치기)는 없습니다. 플러그인은 시드 SQL(242)로 넣고 화면은 읽기만 하기 때문입니다.

함수	하는 일	권한
`list_plugins(db, *, active_only=True)`	종류, 이름 순으로 플러그인 목록. 기본은 쓸 수 있는 것만	로그인한 회원이면 누구나
`get_plugin(db, plugin_id)`	한 건. 없으면 None	위와 같음
`require_plugin(db, plugin_id)`	한 건. 없으면 404 오류를 냅니다	위와 같음
`list_plugin_presets(db, plugin_id, user_id=None)`	시스템 프리셋과 내 프리셋	위와 같음

`user_id` 를 주면 `owner_id IS NULL`(시스템) 이거나 `owner_id = user_id`(내 것) 인 행만 골라 이름 순으로 돌려줍니다. 주지 않으면 시스템 프리셋만 돌려줍니다.

2. 언제 쓰나

쓰는 곳	
`GET /plugins` · `GET /plugins/{id}` · `GET /plugins/{id}/presets` (237)	라우터가 이 함수들을 부릅니다
`track_service.add_instrument` · `add_effect` (234·235)	`require_plugin` 으로 플러그인이 있는지 확인하고 `plugin_type` 을 검사합니다
화면의 악기/이펙트 고르기 목록	`list_plugins` 결과를 그대로 그립니다

3. 코드

목록과 한 건

```
def list_plugins(db: Session, *, active_only: bool = True) -> Sequence[Plugin]:
    stmt = select(Plugin).order_by(Plugin.plugin_type, Plugin.name)
    if active_only:
        stmt = stmt.where(Plugin.is_active == True)  # noqa: E712
```

```
    return db.execute(stmt).scalars().all()


def get_plugin(db: Session, plugin_id: int) -> Plugin | None:
    return db.get(Plugin, plugin_id)


def require_plugin(db: Session, plugin_id: int) -> Plugin:
    p = get_plugin(db, plugin_id)
    if p is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Plugin not found.")
    return p
```

프리셋

```
def list_plugin_presets(
    db: Session, plugin_id: int, user_id: int | None = None
) -> Sequence[PluginPreset]:
    stmt = select(PluginPreset).where(PluginPreset.plugin_id == plugin_id)
    if user_id is not None:
        stmt = stmt.where(or_(PluginPreset.owner_id.is_(None), PluginPreset.owner_id == user_id))
    else:
        stmt = stmt.where(PluginPreset.owner_id.is_(None))
    stmt = stmt.order_by(PluginPreset.name)
    return db.execute(stmt).scalars().all()
```

문법 풀이

식	뜻
`def list_plugins(db, *, active_only=True)`	`*` 뒤 인자는 이름을 붙여야만 넘길 수 있습니다. `list_plugins(db, False)` 같은 실수를 막습니다
`Plugin.is_active == True`	SQL 로 바뀌면 `is_active = 1` 입니다. 파이썬 문법 검사(ruff E712)가 걸려 `noqa` 를 달았습니다
`db.get(Plugin, plugin_id)`	기본 키로 한 건을 찾습니다. 없으면 None 을 돌려줍니다
`raise AppError(ErrorCode.NOT_FOUND, 404, "...")`	오류 코드·상태·메시지 세 가지를 함께 넘겨야 합니다. 화면은 `{"code": "NOT_FOUND"}` 를 받습니다
`or_(A, B)`	SQL 의 `OR` 입니다. 시스템 프리셋과 내 프리셋을 한 번에 고릅니다
`is_(None)`	SQL 의 `IS NULL` 입니다. `== None` 대신 써야 의도가 분명합니다

4. 왜 이렇게

선택	이유
읽기만 두 개	플러그인 정의는 시드로만 바뀝니다. API 로 만들면 화면 구현이 없는 플러그인이 표에 들어갈 수 있습니다
`require_plugin` 을 따로	"없으면 404" 를 여러 곳(237, 234, 235)에서 씁니다. 한 곳에 모아야 메시지와 코드가 어긋나지 않습니다
`user_id` 를 키워드로	라우터가 `current_user.user_id` 를 넘깁니다. 값이 없으면 시스템 프리셋만 나옵니다
프리셋을 이름 순으로	화면의 선택 목록에서 찾기 쉽습니다. `Init` 이 맨 앞에 옵니다

선택	이유
정렬을 `plugin_type` → `name`	화면 목록이 악기부터 보이고, 같은 종류 안에서는 이름 순입니다

5. 시험

`tests/test_plugin_routes.py` 가 다음을 확인합니다.

- 목록 응답의 칸 이름과 `active_only=false` 일 때 꺼진 플러그인도 나오는지
- 상세 응답의 `param_schema` 항목과 없는 번호일 때 404
- 프리셋 목록에 시스템 프리셋과 내가 저장한 프리셋만 나오고 남의 것은 빠지는지
- 로그인하지 않으면 401

Backend(FastAPI/Python) 233 — `track_service` 트랙과 권한 (목록 · 만들기 · 조회 · 고치기 · 지우기)

항목	내용
파일	`studionx/backend/app/services/track_service.py`
작업	S1-09 기본 악기·이펙트: 트랙 다섯 함수와 권한 도우미 (PRD 02 FR-251, FR-252, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	232 플러그인 서비스 · 234 악기 칸

1. 하는 일

트랙 한 줄을 다루는 함수 다섯 개와, 그 아래 모든 작업이 함께 쓰는 권한 도우미 두 개를 둡니다.

함수	하는 일	권한
`_find_track`	트랙과 그 프로젝트의 내 멤버 행을 함께 찾습니다	없거나 남의 것이면 404
`_require_editor_track`	쓰기용. 권한이 없으면 403	OWNER, EDITOR
`list_tracks`	프로젝트의 트랙 목록 (`order_index`, `track_id` 순)	멤버면 누구나
`create_track`	트랙 만들기. 부모가 있으면 같은 프로젝트의 트랙인지 확인	OWNER, EDITOR
`require_track`	트랙 한 건 (조회용)	멤버면 누구나
`update_track`	보낸 칸만 바꿉니다	OWNER, EDITOR
`delete_track`	트랙 지우기. 악기·이펙트·연결선은 DB CASCADE 로 함께 지워집니다	OWNER, EDITOR

권한 규칙은 `project_service`(P1-13)와 같습니다. 조회는 멤버면 누구나이되 **남의 프로젝트는 있는지조차 404 로 숨기고**, 쓰기는 OWNER·EDITOR 만 403 입니다.

2. 언제 쓰나

쓰는 곳	
`GET/POST /projects/{id}/tracks`, `GET/PATCH/DELETE /tracks/{id}` (238)	다섯 라우터가 이 함수들을 부릅니다
`list_instruments` · `list_effects` · `list_links` (234~236)	먼저 `require_track` 으로 조회 권한을 확인합니다
`_require_instrument` · `_require_effect` (234·235)	`_require_editor_track` 으로 쓰기 권한을 확인합니다
프로젝트 만들기 API (P1-13)	MASTER 트랙을 만들 때 `create_track` 을 씁니다 (FR-252)

3. 코드

권한 도우미

```
def _find_track(db: Session, user: User, track_id: int):
    """트랙과 그 프로젝트·내 멤버 행을 함께 찾는다. 없거나 남의 것이면 404."""
    track = db.get(Track, track_id)
    if track is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Track not found.")
    project, member = project_service.get_membership(db, track.project_id, user)
    return track, project, member


def _require_editor_track(db: Session, user: User, track_id: int) -> Track:
    """쓰기가 필요한 작업용: 남의 트랙은 404, 권한이 없으면 403."""
    track, _project, member = _find_track(db, user, track_id)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    return track
```

목록과 만들기

```
def list_tracks(db: Session, user: User, project_id: int) -> Sequence[Track]:
    project_service.get_membership(db, project_id, user)
    stmt = (
        select(Track)
        .where(Track.project_id == project_id)
        .order_by(Track.order_index, Track.track_id)
    )
    return db.execute(stmt).scalars().all()


def create_track(db, user, project_id, *, name, track_type=TrackType.MIDI, color="#7C5CFF",
                 volume_db=Decimal("0.00"), pan=Decimal("0.00"), is_muted=False, is_solo=False,
                 is_enabled=True, routing_mode=RoutingMode.CHAIN, order_index=0,
                 parent_track_id=None) -> Track:
    project, member = project_service.get_membership(db, project_id, user)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    if parent_track_id:
        parent = db.get(Track, parent_track_id)
        if parent is None or parent.project_id != project.project_id:
            raise AppError(ErrorCode.BAD_REQUEST, 400, "Invalid parent track.")
    track = Track(project_id=project.project_id, name=name, track_type=track_type, ...)
    db.add(track)
    db.commit()
    db.refresh(track)
    return track
```

고치기와 지우기

```
def update_track(db: Session, user: User, track_id: int, **kwargs) -> Track:
    track = _require_editor_track(db, user, track_id)
    for key, value in kwargs.items():
        if value is not None and hasattr(track, key):
            setattr(track, key, value)
    db.commit()
    db.refresh(track)
    return track
```

```
def delete_track(db: Session, user: User, track_id: int) -> None:
    track = _require_editor_track(db, user, track_id)
    db.delete(track)
    db.commit()
```

문법 풀이

식	뜻
`db.get(Track, track_id)`	기본 키로 한 건을 찾습니다. 없으면 None
`project_service.get_membership(db, project_id, user)`	프로젝트와 내 멤버 행을 돌려줍니다. 남의 것이면 여기서 404 가 납니다
`require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	둘 중 하나라도 아니면 403 입니다. 역할이 여러 개일 때 이렇게 나열합니다
`_project`, `_member`	안 쓰는 값이라는 표시입니다. `_find_track` 이 셋을 돌려주므로 자리만 받습니다
`**kwargs` 로 받고 `hasattr` 로 거름	보낸 칸만 바꾸는 방식입니다. 모델에 없는 이름이 와도 무시합니다
`if value is not None`	None 을 보내도 그대로 두는 규칙입니다. 라우터가 안 보낸 칸도 None 이라 함께 걸러집니다
`db.commit()` 뒤 `db.refresh()`	실제 DB 에 남기고, DB 가 채운 값(기본값·`updated_at`)까지 다시 읽어 응답에 담습니다

4. 왜 이렇게

선택	이유
인자를 `(db, user, ...)` 로 통일	P1-13 프로젝트 서비스와 같은 모양입니다. 서비스마다 순서가 다르면 실수가 납니다
남의 트랙을 404 로	403 을 돌려주면 "그 번호의 트랙이 있다" 는 사실이 새어 나갑니다. 프로젝트 서비스와 같은 규칙입니다
쓰기 뒤 `commit`	`get_db` 는 commit 을 하지 않습니다. 서비스가 직접 커밋하지 않으면 응답은 나가지만 DB 에 남지 않습니다
목록 정렬에 `track_id` 를 덧붙임	순서 번호가 같을 때(새로 만든 직후) 화면 순서가 흔들리지 않게 합니다
부모 트랙을 같은 프로젝트로 제한	다른 프로젝트의 트랙을 부모로 넣으면 그룹 접기/펼치기가 꼬입니다 (FR-252)
삭제를 DB 에 맡김	악기·이펙트·연결선 세 표가 `ON DELETE CASCADE` 입니다. 앱이 빠뜨릴 일이 없습니다

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 목록 순서(`order_index`, `track_id`), 만들기 201과 기본값
- 색 형식 오류와 볼륨 범위 초과는 422
- 고치기 응답(이름·볼륨·`routing_mode` = `PATCHER`), 지운 뒤 404
- 남의 프로젝트 트랙은 조회·목록·만들기 모두 404
- VIEWER 는 조회 200, 쓰기(만들기·고치기·지우기) 403

Backend(FastAPI/Python) 234 — `track_service` 악기 칸 (칸 번호 확인과 네 함수)

항목	내용
파일	`studionx/backend/app/services/track_service.py`
작업	S1-09 기본 악기·이펙트: 악기 칸 읽기·붙이기·고치기·떼기 (PRD 02 FR-245, FR-264)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	233 트랙과 권한 · 235 이펙트 칸

1. 하는 일

트랙 안의 악기 칸(`track_instruments`)을 다루는 함수 다섯 개를 둡니다.

함수	하는 일	권한
`_require_free_slot`	같은 트랙의 같은 칸에 이미 행이 있는지 확인하고, 있으면 400	쓰기 준비
`list_instruments`	트랙의 악기 칸 목록 (칸 번호 순)	멤버면 누구나
`add_instrument`	악기 한 칸 붙이기. `INSTRUMENT` 플러그인만 가능	OWNER, EDITOR
`update_instrument`	파라미터, 볼륨, 팬, 솔로, 켜기, 칸 번호 고치기	OWNER, EDITOR
`remove_instrument`	악기 칸 떼기	OWNER, EDITOR

악기 칸은 **가상악기(INSTRUMENT)만** 들어갑니다. 이펙트를 넣으려 하면 400 입니다. 종류를 섞으면 소리 처리 순서가 꼬이기 때문입니다.

2. 언제 쓰나

쓰는 곳	
`GET/POST /tracks/{id}/instruments`, `PATCH/DELETE /tracks/{id}/instruments/{iid}` (239)	네 라우터가 이 함수들을 부릅니다
화면의 [Instrument] 칸	악기를 붙이고 파라미터를 저장할 때 부릅니다 (FR-262·264)
S1-10 저장·불러오기	트랙의 악기 구성을 함께 저장하고 되살립니다 (FR-254)

3. 코드

칸 번호 확인 (S1-09 에 추가한 안전장치)

```
def _require_free_slot(
    db: Session, model, track_id: int, slot_index: int, what: str, exclude_id: int | None = None
) -> None:
    """같은 트랙의 같은 칸에는 한 행만 넣을 수 있다 (DB UNIQUE (track_id, slot_index)).
```

```
    안 막으면 DB 가 IntegrityError 를 뱉어 500 이 된다. 화면이 이미 찬 칸에 넣으려 하면
    400 으로 알려 주어, 다른 칸 번호를 골라 다시 보내게 한다 (FR-263 순서 편집).
    exclude_id 는 고쳐지는 칸 자신이다 (자기 자리에 다시 넣는 것은 막지 않는다).
    """
    pk_name = model.__mapper__.primary_key[0].name
    stmt = (
        select(func.count())
        .select_from(model)
        .where(model.track_id == track_id, model.slot_index == slot_index)
    )
    if exclude_id is not None:
        stmt = stmt.where(getattr(model, pk_name) != exclude_id)
    if db.scalar(stmt):
        raise AppError(
            ErrorCode.BAD_REQUEST, 400, f"Slot {slot_index} is already used by another {what}."
        )
```

목록과 한 칸 찾기

```
def list_instruments(db: Session, user: User, track_id: int) -> Sequence[TrackInstrument]:
    track = require_track(db, user, track_id)
    stmt = (
        select(TrackInstrument)
        .where(TrackInstrument.track_id == track.track_id)
        .order_by(TrackInstrument.slot_index, TrackInstrument.track_instrument_id)
    )
    return db.execute(stmt).scalars().all()


def _require_instrument(db: Session, user: User, track_id: int, instrument_id: int):
    inst = db.get(TrackInstrument, instrument_id)
    if inst is None or inst.track_id != track_id:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Track instrument not found.")
    _require_editor_track(db, user, track_id)
    return inst
```

붙이기 · 고치기 · 떼기

```
def add_instrument(db, user, track_id, *, plugin_id, slot_index=0, params=None, ...):
    _require_editor_track(db, user, track_id)
    plugin = plugin_service.require_plugin(db, plugin_id)
    if plugin.plugin_type != PluginType.INSTRUMENT:
        raise AppError(ErrorCode.BAD_REQUEST, 400, "Only INSTRUMENT plugins can be added to instrument
s.")
    _require_free_slot(db, TrackInstrument, track_id, slot_index, "instrument")
    inst = TrackInstrument(track_id=track_id, plugin_id=plugin_id, slot_index=slot_index,
                           params=params or {}, volume_db=volume_db, pan=pan,
                           is_solo=is_solo, is_enabled=is_enabled)
    db.add(inst)
    db.commit()
    db.refresh(inst)
    return inst


def update_instrument(db, user, track_id, instrument_id, **kwargs):
    inst = _require_instrument(db, user, track_id, instrument_id)
```

```
    if "slot_index" in kwargs and kwargs["slot_index"] is not None:
        _require_free_slot(db, TrackInstrument, track_id, kwargs["slot_index"], "instrument",
                           exclude_id=inst.track_instrument_id)
    for key, value in kwargs.items():
        if value is not None and hasattr(inst, key):
            setattr(inst, key, value)
    db.commit()
    db.refresh(inst)
    return inst
```

문법 풀이

식	뜻
`model.__mapper__.primary_key[0].name`	그 모델의 기본 키 칸 이름을 꺼냅니다. 악기(`track_instrument_id`)와 이펙트(`track_plugin_id`)가 달라 이름을 받아 씁니다
`getattr(model, pk_name) != exclude_id`	"나 자신은 빼고" 세는 조건입니다
`db.scalar(select(func.count())...)`	개수를 하나의 값으로 받습니다. 0 이면 비어 있습니다
`f"Slot {slot_index} is already used..."`	오류 메시지에 칸 번호를 넣습니다. 화면이 어느 칸이 문제인지 알 수 있습니다
`.order_by(slot_index, track_instrument_id)`	칸 번호 순으로, 같으면 번호 순으로 정렬합니다
`params or {}`	안 보내면 빈 사전을 넣습니다. 악기의 파라미터 칸은 NULL 도 되지만, 화면이 바로 쓸 수 있게 빈 사전으로 둡니다
`inst.track_id != track_id`	경로의 트랙과 다른 트랙의 악기는 404 입니다. 주소를 손으로 바꿔 남의 악기를 고치는 일을 막습니다

4. 왜 이렇게

선택	이유
목록을 관계 대신 쿼리로	같은 요청 안에서 지우고 바로 다시 읽을 때, 관계(`track.instruments`)에는 지워진 행이 남아 있습니다(DB 삭제를 CASCADE 에 맡겨서). 쿼리로 읽으면 늘 실제 DB 값입니다
칸 번호를 미리 확인	DB 에는 `UNIQUE (track_id, slot_index)` 가 있어 위반하면 `IntegrityError`(500)가 납니다. 400 으로 알려 주면 화면이 다음 칸 번호를 골라 다시 보낼 수 있습니다 (FR-263)
`exclude_id` 로 자기 자리 허용	"볼륨만 바꾸기" 처럼 같은 칸을 그대로 다시 보내는 요청이 400 이 되면 안 됩니다
붙이기에서 플러그인 종류 검사	악기 자리에 이펙트가 들어가면 소리 처리가 뒤섞입니다. 400 으로 막습니다 (FR-245)
권한 검사를 `_require_editor_track` 한 곳으로	경로마다 같은 규칙을 다시 쓰면 한 곳만 빠뜨려도 구멍이 납니다

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 붙이기 201과 응답 칸 이름, 이펙트를 악기 자리에 넣으면 400, 없는 플러그인이면 404
- 목록이 칸 번호 순이고, 지운 뒤에는 빈 목록
- 고치기(볼륨, 켜기)와 떼기 204
- 같은 칸에 두 번 넣으면 400, 옮길 칸이 이미 차 있으면 400, 자기 자리로는 200
- 로그인하지 않으면 401

Backend(FastAPI/Python) 235 — `track_service` 이펙트 칸 (체인 순서와 갈래 자리)

항목	내용
파일	`studionx/backend/app/services/track_service.py`
작업	S1-09 기본 악기·이펙트: 이펙트 칸 읽기·붙이기·고치기·떼기 (PRD 02 FR-263, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	234 악기 칸 · 236 연결선

1. 하는 일

트랙 안의 이펙트 칸(`track_plugins`)을 다루는 함수 다섯 개를 둡니다.

함수	하는 일	권한
`list_effects`	트랙의 이펙트 목록 (칸 번호 순 = 체인 순서)	멤버면 누구나
`add_effect`	이펙트 한 칸 붙이기. `EFFECT` 플러그인만 가능	OWNER, EDITOR
`update_effect`	칸 번호(순서), 카드 자리(`node_x`·`node_y`), 파라미터, 바이패스 고치기	OWNER, EDITOR
`remove_effect`	이펙트 칸 떼기. 그 칸에 이어진 연결선도 DB CASCADE 로 함께 지워집니다	OWNER, EDITOR
`_require_effect`	경로의 트랙에 속한 이펙트인지 확인하고 쓰기 권한까지 검사	OWNER, EDITOR

이펙트 칸은 **EFFECT 플러그인만** 들어갑니다. `slot_index` 가 소리가 지나가는 순서이고, `node_x`·`node_y` 는 갈래(패처) 화면에서 카드를 놓은 자리입니다 (FR-269).

2. 언제 쓰나

쓰는 곳	
`GET/POST /tracks/{id}/plugins`, `PATCH/DELETE /tracks/{id}/plugins/{pid}` (240)	네 라우터가 이 함수들을 부릅니다
화면의 [Effects] 칸	체인 카드를 붙이고 끌어 순서를 바꾸고 파라미터를 저장할 때 부릅니다 (FR-263)
`add_link` (236)	연결선이 가리키는 이펙트가 같은 트랙의 것인지 확인할 때 씁니다
S1-10 저장·불러오기	이펙트 구성과 순서, 갈래 자리를 함께 저장하고 되살립니다 (FR-254)

3. 코드

목록과 한 칸 찾기

```
def list_effects(db: Session, user: User, track_id: int) -> Sequence[TrackPlugin]:
    """트랙의 이펙트 칸 목록 (slot_index 순 = 체인 순서). 멤버면 누구나 (FR-263)."""
    track = require_track(db, user, track_id)
```

```
    stmt = (
        select(TrackPlugin)
        .where(TrackPlugin.track_id == track.track_id)
        .order_by(TrackPlugin.slot_index, TrackPlugin.track_plugin_id)
    )
    return db.execute(stmt).scalars().all()


def _require_effect(db: Session, user: User, track_id: int, track_plugin_id: int):
    tp = db.get(TrackPlugin, track_plugin_id)
    if tp is None or tp.track_id != track_id:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Track effect not found.")
    _require_editor_track(db, user, track_id)
    return tp
```

붙이기

```
def add_effect(db, user, track_id, *, plugin_id, slot_index, node_x=None, node_y=None,
               params=None, is_bypassed=False):
    """트랙에 이펙트 한 칸 붙이기 (FR-245·263). OWNER·EDITOR 만, EFFECT 만 가능."""
    _require_editor_track(db, user, track_id)
    plugin = plugin_service.require_plugin(db, plugin_id)
    if plugin.plugin_type != PluginType.EFFECT:
        raise AppError(ErrorCode.BAD_REQUEST, 400, "Only EFFECT plugins can be added to effects chain.")
    _require_free_slot(db, TrackPlugin, track_id, slot_index, "effect")
    tp = TrackPlugin(track_id=track_id, plugin_id=plugin_id, slot_index=slot_index,
                     node_x=node_x, node_y=node_y, params=params or {}, is_bypassed=is_bypassed)
    db.add(tp)
    db.commit()
    db.refresh(tp)
    return tp
```

고치기 (순서 바꾸기 포함)

```
def update_effect(db, user, track_id, track_plugin_id, **kwargs):
    tp = _require_effect(db, user, track_id, track_plugin_id)
    if "slot_index" in kwargs and kwargs["slot_index"] is not None:
        _require_free_slot(db, TrackPlugin, track_id, kwargs["slot_index"], "effect",
                           exclude_id=tp.track_plugin_id)
    for key, value in kwargs.items():
        if value is not None and hasattr(tp, key):
            setattr(tp, key, value)
    db.commit()
    db.refresh(tp)
    return tp
```

문법 풀이

식	뜻
`slot_index: int` (기본값 없음)	이펙트는 순서가 있어야 하므로 붙일 때 반드시 받습니다. 악기 칸과 다른 점입니다
`_require_free_slot(db, TrackPlugin, ...)`	악기와 같은 도우미를 이펙트 모델로 부릅니다 (234 §3)

식	뜻
`exclude_id=tp.track_plugin_id`	자기 자리로 다시 보내는 요청은 통과시킵니다
`"slot_index" in kwargs`	순서를 바꾸는 요청일 때만 칸 검사를 합니다. 파라미터만 바꿀 때는 검사하지 않습니다
`params or {}`	붙일 때 파라미터를 안 보내면 빈 사전입니다. 나중에 화면이 기본값으로 채웁니다
`hasattr(tp, key)`	모델에 있는 칸만 바꿉니다. `track_id` 같은 칸을 손으로 바꾸는 요청을 막습니다

4. 왜 이렇게

선택	이유
칸 번호가 곧 체인 순서	CHAIN 방식은 소리가 `slot_index` 차례로 지나갑니다. 따로 순서 칸을 두지 않아도 됩니다 (FR-263·267)
순서 바꾸기를 `PATCH` 하나로	카드를 끌어 옮기는 일은 "그 카드의 칸 번호를 바꾼다" 로 표현됩니다. 화면은 옮긴 카드 한 장만 보내면 됩니다
이미 찬 칸이면 400	두 이펙트가 같은 번호를 가지면 순서를 정할 수 없습니다. DB UNIQUE 를 서비스가 먼저 확인합니다 (234 §4)
자리(`node_x`·`node_y`)를 그대로 저장	갈래 화면을 다시 열어도 카드가 놓인 자리가 같아야 합니다 (FR-269)
떼면 연결선도 함께	선만 남으면 없는 카드를 가리킵니다. DB CASCADE 로 함께 지웁니다
`is_bypassed` 를 따로	소리 처리만 잠시 건너뜁니다. 카드는 그대로 남아 순서와 설정을 지킵니다

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 붙이기 201과 응답 칸 이름, `node_x` 저장, `is_bypassed` 기본값 false
- 악기를 이펙트 자리에 넣으면 400
- 목록 개수와 고치기(바이패스, 칸 번호 2로 옮기기)
- 같은 칸 두 번은 400, 남이 쓰는 칸으로 옮기기도 400
- 떼기 204

> 슬롯을 서로 맞바꾸는 화면 동작(예: 1번과 2번 교환)은 지금 API 로는 한 번에 안 됩니다. 한 카드를 먼저 빈 칸(예: 100번)으로 옮기고, 다른 카드를 원래 자리로 옮긴 뒤, 첫 카드를 마지막 자리로 옮기는 세 번의 `PATCH` 로 처리합니다 (FR-263 순서 편집).

Backend(FastAPI/Python) 236 — `track_service` 연결선 (갈래 나누기와 합치기)

항목	내용
파일	`studionx/backend/app/services/track_service.py`
작업	S1-09 기본 악기·이펙트: 이펙트 연결선 읽기·긋기·지우기 (PRD 02 FR-265, FR-267, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	235 이펙트 칸 · 237 플러그인 API

1. 하는 일

갈래(패처) 구성에서 이펙트 카드를 잇는 선(`track_plugin_links`)을 다루는 함수 세 개를 둡니다.

함수	하는 일	권한
`list_links`	트랙의 연결선 목록 (번호 순)	멤버면 누구나
`add_link`	연결선 한 줄 긋기. 가리키는 이펙트가 같은 트랙의 것인지 확인	OWNER, EDITOR
`remove_link`	연결선 지우기	OWNER, EDITOR

`from` 하나에서 여러 `to` 로 가면 갈래가 나뉘고, 여러 `from` 이 `to` 하나로 모이면 병합입니다. `from` 이 비어 있으면 트랙 입력(소스), `to` 가 비어 있으면 트랙 출력입니다 (FR-267).

2. 언제 쓰나

쓰는 곳	
`GET/POST /tracks/{id}/links`, `DELETE /tracks/{id}/links/{lid}` (241)	세 라우터가 이 함수들을 부릅니다
화면의 갈래(패처) 보드	카드를 이어 갈래를 만들고, 병합하고, 선을 지울 때 부릅니다 (FR-265)
S1-10 저장·불러오기	갈래 구성을 함께 저장하고 되살립니다 (FR-254)

3. 코드

목록

```
def list_links(db: Session, user: User, track_id: int) -> Sequence[TrackPluginLink]:
    """트랙의 연결선 목록 (FR-267). 멤버면 누구나."""
    track = require_track(db, user, track_id)
    stmt = (
        select(TrackPluginLink)
        .where(TrackPluginLink.track_id == track.track_id)
        .order_by(TrackPluginLink.link_id)
    )
    return db.execute(stmt).scalars().all()
```

긋기

```
def add_link(db, user, track_id, *, from_track_plugin_id=None, from_port=0,
             to_track_plugin_id=None, to_port=0, gain_db=Decimal("0.00")):
    """갈래 연결선 붙이기 (FR-267·269). OWNER·EDITOR 만."""
    track = _require_editor_track(db, user, track_id)
    for plugin_id in (from_track_plugin_id, to_track_plugin_id):
        if plugin_id is None:
            continue
        tp = db.get(TrackPlugin, plugin_id)
        if tp is None or tp.track_id != track.track_id:
            raise AppError(ErrorCode.NOT_FOUND, 404, "Track effect not found.")
    link = TrackPluginLink(track_id=track.track_id, from_track_plugin_id=from_track_plugin_id,
                           from_port=from_port, to_track_plugin_id=to_track_plugin_id,
                           to_port=to_port, gain_db=gain_db)
    db.add(link)
    db.commit()
    db.refresh(link)
    return link
```

지우기

```
def remove_link(db: Session, user: User, track_id: int, link_id: int) -> None:
    """연결선 떼기 (FR-267). OWNER·EDITOR 만."""
    link = db.get(TrackPluginLink, link_id)
    if link is None or link.track_id != track_id:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Track plugin link not found.")
    _require_editor_track(db, user, track_id)
    db.delete(link)
    db.commit()
```

문법 풀이

식	뜻
`for plugin_id in (from, to)`	두 끝을 같은 코드로 검사합니다. 이펙트가 없거나 남의 트랙 것이면 404 입니다
`if plugin_id is None: continue`	트랙 입력이나 출력(비어 있는 끝)은 검사할 이펙트가 없습니다
`db.get(TrackPlugin, plugin_id)`	이펙트 칸이 실제 있는지 봅니다
`tp.track_id != track.track_id`	다른 트랙의 이펙트를 가리키는 선은 404 입니다. 갈래는 트랙 안에서만 이뤄집니다
`external` 이름 대신 `from·to`	`from` 이 파이썬 예약어라 인자 이름에 쓸 수 없습니다. 그래서 `from_track_plugin_id` 처럼 표의 칸 이름을 그대로 씁니다
`.order_by(link_id)`	만든 순서대로 돌려줍니다. 화면이 선을 늘 같은 차례로 그립니다

4. 왜 이렇게

선택	이유
선을 지우고 다시 긋는 방식	지금 화면에는 선 하나를 고치는 동작이 없습니다. 필요해지면 `PATCH` 를 더합니다 (`TrackPluginLinkUpdateRequest` 는 이미 준비되어 있습니다)
양 끝이 한 트랙 안	트랙 사이의 연결은 다른 표(`track_connections`, DB.md Phase 2)가 맡습니다. 이 표는 트랙 안 갈래 전용입니다 (FR-266)
빈 끝을 허용	트랙 입력과 출력도 같은 표로 담아야 화면이 선을 다 그릴 수 있습니다 (FR-267)
이펙트를 지우면 선도 삭제	DB 의 `ON DELETE CASCADE` 가 처리합니다. 없는 카드를 가리키는 선이 남지 않습니다
`gain_db` 를 선에 둠	갈래마다 세기를 다르게 보낼 수 있습니다 (FR-266)

5. 시험

`tests/test_track_routes.py` 의 `test_patcher_links_crud` 가 트랙 입력에서 이펙트 A 로 가는 선, 이펙트 A 에서 이펙트 B 로 가는 선을 만들고, 목록 개수와 `gain_db` 저장, 지우기를 확인합니다. 다른 트랙의 이펙트를 가리키면 404 입니다.

Backend(FastAPI/Python) 237 — 플러그인 API (`/plugins`)

항목	내용
파일	`studionx/backend/app/api/v1/plugins.py`
작업	S1-09 기본 악기·이펙트: 플러그인 조회 경로 세 개 (PRD 02 FR-241, FR-244, FR-246)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	236 연결선 · 238 트랙 API

1. 하는 일

플러그인을 읽는 경로 세 개를 엽니다. 등록은 `router.py` 에서 `prefix="/plugins"` 로 붙습니다.

경로	하는 일	상태 코드
`GET /plugins`	목록. `?active_only=false` 면 못 쓰는 것도 포함	200, 401
`GET /plugins/{plugin_id}`	한 건	200, 401, 404
`GET /plugins/{plugin_id}/presets`	시스템 프리셋과 내 프리셋	200, 401, 404

세 경로 모두 로그인(`CurrentUser`)이 필요합니다. 라우터는 요청·응답 모양과 상태 코드만 정하고, 찾기와 오류는 `plugin_service`(232)에 맡깁니다.

2. 언제 쓰나

쓰는 곳	
화면의 악기/이펙트 고르기 창	`GET /plugins?active_only=true` 로 목록을 그립니다 (FR-244)
파라미터 그리기	`GET /plugins/{id}` 의 `param_schema` 를 읽어 노브를 만듭니다 (FR-246)
프리셋 단추	`GET /plugins/{id}/presets` 로 고를 수 있는 프리셋을 채웁니다 (FR-242)

3. 코드

목록

```
@router.get("", response_model=PluginListResponse, responses={401: {"model": ErrorResponse}})
def list_plugins(
    db: Db,
    current_user: CurrentUser,
    active_only: Annotated[bool, Query(description="활성 플러그인만")] = True,
):
    items = plugin_service.list_plugins(db, active_only=active_only)
    return PluginListResponse(items=[PluginResponse.model_validate(p) for p in items])
```

한 건과 프리셋

```
@router.get("/{plugin_id}", response_model=PluginResponse,
            responses={401: {"model": ErrorResponse}, 404: {"model": ErrorResponse}})
```

```
def get_plugin(plugin_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    p = plugin_service.require_plugin(db, plugin_id)
    return PluginResponse.model_validate(p)


@router.get("/{plugin_id}/presets", response_model=list[PluginPresetResponse],
            responses={401: {"model": ErrorResponse}, 404: {"model": ErrorResponse}})
def list_plugin_presets(plugin_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    p = plugin_service.require_plugin(db, plugin_id)
    presets = plugin_service.list_plugin_presets(db, p.plugin_id, user_id=current_user.user_id)
    return [PluginPresetResponse.model_validate(pr) for pr in presets]
```

문법 풀이

식	뜻
`@router.get("")`	앞 경로가 `"/plugins"` 뿐이라 빈 문자열을 씁니다. `"/"` 로 쓰면 뒤에 슬래시가 붙어 307 리다이렉트가 납니다
`db: Db`, `current_user: CurrentUser`	`Annotated` 로 묶은 의존성입니다. 함수만 적으면 FastAPI 가 세션과 로그인 회원을 넣어 줍니다
`Annotated[bool, Query(...)] = True`	주소의 `?active_only=false` 를 받습니다. 안 주면 True
`Annotated[int, Path(ge=1)]`	경로 값이 1 이상이어야 합니다. 0 이면 422 입니다
`responses={401: {"model": ErrorResponse}}`	문서(Swagger)에 오류 모양을 보여 줍니다. 실제 응답은 `AppError` 처리기가 만듭니다
`PluginResponse.model_validate(p)`	ORM 객체를 응답 모델로 바꿉니다. 목록은 `items=[...]` 로 감쌉니다

4. 왜 이렇게

선택	이유
프리셋 경로도 플러그인 존재를 먼저 확인	없는 플러그인의 프리셋은 빈 목록이 아니라 404 가 맞습니다. 화면이 잘못된 번호를 바로 알아챕니다
`current_user` 를 안 쓰는 목록에도 받음	로그인하지 않으면 401 을 내야 합니다. 안 받으면 누구나 플러그인 목록을 볼 수 있습니다
쓰기 경로 없음	플러그인 정의는 시드(242)가 넣습니다. 화면에서 만들면 `param_schema` 없는 플러그인이 생깁니다
목록 응답을 `{items: [...]}` 로 감쌈	플러그인 목록은 개수(`total`)를 함께 주는 편이 화면에 낫습니다 (232 와 같은 모양)

5. 시험

`tests/test_plugin_routes.py` 가 다음을 확인합니다.

- 로그인 뒤 목록 200과 `items` 배열, `active_only=false` 일 때 비활성 플러그인 포함
- `param_schema` 가 비어 있지 않고 종류(`plugin_type`)가 `INSTRUMENT`·`EFFECT` 중 하나
- 없는 번호일 때 404, 번호가 0 일 때 422
- 로그인하지 않으면 목록·한 건·프리셋 모두 401 (`test_presets_requires_login`)

Backend(FastAPI/Python) 238 — 트랙 API (목록 · 만들기 · 조회 · 고치기 · 지우기)

항목	내용
파일	`studionx/backend/app/api/v1/tracks.py`
작업	S1-09 기본 악기·이펙트: 트랙 경로 다섯 개와 공통 오류 모양 (PRD 02 FR-251, FR-252, FR-253)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	237 플러그인 API · 239 악기 API

1. 하는 일

트랙을 다루는 경로 다섯 개를 엽니다. 이 파일 하나가 트랙과 그 안의 악기·이펙트·연결선 경로를 모두 담고 있습니다(239~241). 등록은 `router.py` 에서 **앞 경로 없이**(`prefix` 없이) 붙여, 주소가 `/projects/{id}/tracks` 와 `/tracks/{id}` 가 되게 했습니다.

경로	하는 일	상태 코드
`GET /projects/{project_id}/tracks`	프로젝트의 트랙 목록	200, 401, 404
`POST /projects/{project_id}/tracks`	트랙 만들기	201, 400, 401, 403, 404
`GET /tracks/{track_id}`	트랙 한 건	200, 401, 404
`PATCH /tracks/{track_id}`	보낸 칸만 고치기	200, 400, 401, 403, 404
`DELETE /tracks/{track_id}`	트랙 지우기	204, 401, 403, 404

2. 언제 쓰나

쓰는 곳	
화면의 트랙 추가 단추	`POST` 로 트랙을 만들고 헤더를 그립니다 (FR-253)
프로젝트 열기	`GET /projects/{id}/tracks` 로 트랙 목록과 각 트랙의 상태를 읽습니다
트랙 헤더 믹서	이름·색·볼륨·팬·뮤트·솔로를 `PATCH` 한 번으로 보냅니다
트랙 지우기	`DELETE` 로 트랙과 그 안의 악기·이펙트·연결선을 함께 지웁니다

3. 코드

공통 오류 모양

```
router = APIRouter(tags=["tracks"])

NOT_FOUND_404 = {404: {"model": ErrorResponse}}
WRITE_RESPONSES = {
    400: {"model": ErrorResponse},
    401: {"model": ErrorResponse},
    403: {"model": ErrorResponse},
```

```
    404: {"model": ErrorResponse},
}
```

목록과 만들기

```
@router.get("/projects/{project_id}/tracks", response_model=TrackListResponse, responses={...})
def list_tracks(project_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    items = track_service.list_tracks(db, current_user, project_id)
    return TrackListResponse(
        items=[TrackResponse.model_validate(t) for t in items], total=len(items)
    )


@router.post("/projects/{project_id}/tracks", response_model=TrackResponse,
             status_code=status.HTTP_201_CREATED, responses=WRITE_RESPONSES)
def create_track(project_id: Annotated[int, Path(ge=1)], req: TrackCreateRequest,
                 db: Db, current_user: CurrentUser):
    track = track_service.create_track(
        db, current_user, project_id,
        name=req.name, track_type=req.track_type, color=req.color, volume_db=req.volume_db,
        pan=req.pan, is_muted=req.is_muted, is_solo=req.is_solo, is_enabled=req.is_enabled,
        routing_mode=req.routing_mode, order_index=req.order_index, parent_track_id=req.parent_track_id,
    )
    return TrackResponse.model_validate(track)
```

조회 · 고치기 · 지우기

```
@router.get("/tracks/{track_id}", response_model=TrackResponse, responses={...})
def get_track(track_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    track = track_service.require_track(db, current_user, track_id)
    return TrackResponse.model_validate(track)


@router.patch("/tracks/{track_id}", response_model=TrackResponse, responses=WRITE_RESPONSES)
def update_track(track_id: Annotated[int, Path(ge=1)], req: TrackUpdateRequest,
                 db: Db, current_user: CurrentUser):
    data = req.model_dump(exclude_unset=True)
    track = track_service.update_track(db, current_user, track_id, **data)
    return TrackResponse.model_validate(track)


@router.delete("/tracks/{track_id}", status_code=status.HTTP_204_NO_CONTENT, responses={...})
def delete_track(track_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    track_service.delete_track(db, current_user, track_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

문법 풀이

식	뜻
`WRITE_RESPONSES` 를 만들어 재사용	쓰기 경로 넷이 같은 오류 목록(400·401·403·404)을 씁니다. 한 곳만 고치면 모두 바뀝니다
`{**NOT_FOUND_404}`	사전을 펼쳐 넣습니다. 404 만 필요한 경로에 씁니다
`status_code=status.HTTP_201_CREATED`	만들기 성공은 201 입니다. 화면이 "새로 만들어졌다" 를 구분합니다

식	뜻
`status.HTTP_204_NO_CONTENT` + `Response(...)`	지우기 성공은 본문 없는 204 입니다. `Response` 를 돌려주면 본문이 나가지 않습니다
`req.model_dump(exclude_unset=True)`	요청에 **실제로 들어온** 칸만 사전으로 꺼냅니다. 안 보낸 칸은 빠져 서비스가 건드리지 않습니다
`**data`	사전을 이름 붙은 인자로 풀어 넘깁니다. 서비스의 `**kwargs` 가 받습니다
`total=len(items)`	목록을 한 번 읽고 개수를 셉니다. 트랙 목록은 작아 별도 `COUNT` 쿼리가 필요 없습니다

4. 왜 이렇게

선택	이유
트랙 전 경로를 한 파일에	주소가 트랙을 중심으로 이어집니다(`/tracks/{id}/instruments`). 파일을 나누면 공통 상수를 여러 곳에 두게 됩니다
`prefix` 없이 등록	목록·만들기는 프로젝트 아래(`/projects/{id}/tracks`), 나머지는 트랙 아래(`/tracks/{id}`)라 한 prefix 로 묶을 수 없습니다
204 로 지우기	지운 뒤 돌려줄 내용이 없습니다. 화면은 성공 여부만 알면 됩니다
서비스에 권한을 모두 맡김	라우터에 권한 검사가 섞이면 경로마다 빠뜨릴 수 있습니다. 상태 코드만 정합니다 (233)
403 과 404 를 함께 문서에	남의 프로젝트 트랙은 404, 내 프로젝트지만 VIEWER 면 403 입니다

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 목록이 `order_index`, `track_id` 순이고 `total` 이 개수와 같음
- 만들기 201과 기본값, 없는 프로젝트 404, VIEWER 403
- 한 건 조회·고치기·지우기(204, 지운 뒤 404)
- 남의 프로젝트 트랙 조회·목록은 404
- 8개 경로가 실제로 등록되었는지 (`test_track_routes_registered`: 트랙 2, 악기 2, 이펙트 2, 연결선 2)

Backend(FastAPI/Python) 239 — 악기 API (목록 · 붙이기 · 고치기 · 떼기)

항목	내용
파일	`studionx/backend/app/api/v1/tracks.py` (악기 경로)
작업	S1-09 기본 악기·이펙트: 트랙 안 악기 칸 경로 네 개 (PRD 02 FR-245, FR-262, FR-264)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	238 트랙 API · 240 이펙트 API

1. 하는 일

트랙 안의 악기 칸을 다루는 경로 네 개를 엽니다.

경로	하는 일	상태 코드
`GET /tracks/{track_id}/instruments`	악기 칸 목록 (칸 번호 순)	200, 401, 404
`POST /tracks/{track_id}/instruments`	악기 붙이기	201, 400, 401, 403, 404
`PATCH /tracks/{track_id}/instruments/{instrument_id}`	파라미터·볼륨·팬·솔로·켜기·칸 번호 고치기	200, 400, 401, 403, 404
`DELETE /tracks/{track_id}/instruments/{instrument_id}`	악기 떼기	204, 401, 403, 404

2. 언제 쓰나

쓰는 곳	
화면의 [Instrument] 칸	`GET` 으로 현재 악기를 그리고, `POST` 로 가상악기를 붙입니다
파라미터 편집	노브를 돌린 값을 `PATCH` 로 모아 보냅니다 (FR-262)
악기 떼기	`DELETE` 로 칸을 비웁니다

3. 코드

목록과 붙이기

```
@router.get("/tracks/{track_id}/instruments", response_model=list[TrackInstrumentResponse],
            responses={...})
def list_instruments(track_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    instruments = track_service.list_instruments(db, current_user, track_id)
    return [TrackInstrumentResponse.model_validate(i) for i in instruments]


@router.post("/tracks/{track_id}/instruments", response_model=TrackInstrumentResponse,
             status_code=status.HTTP_201_CREATED, responses=WRITE_RESPONSES)
def add_instrument(track_id: Annotated[int, Path(ge=1)], req: TrackInstrumentCreateRequest,
```

```
                    db: Db, current_user: CurrentUser):
    inst = track_service.add_instrument(
        db, current_user, track_id,
        plugin_id=req.plugin_id, slot_index=req.slot_index, params=req.params,
        volume_db=req.volume_db, pan=req.pan, is_solo=req.is_solo, is_enabled=req.is_enabled,
    )
    return TrackInstrumentResponse.model_validate(inst)
```

고치기와 떼기

```
@router.patch("/tracks/{track_id}/instruments/{instrument_id}",
              response_model=TrackInstrumentResponse, responses=WRITE_RESPONSES)
def update_instrument(track_id: Annotated[int, Path(ge=1)],
                      instrument_id: Annotated[int, Path(ge=1)],
                      req: TrackInstrumentUpdateRequest, db: Db, current_user: CurrentUser):
    data = req.model_dump(exclude_unset=True)
    inst = track_service.update_instrument(db, current_user, track_id, instrument_id, **data)
    return TrackInstrumentResponse.model_validate(inst)


@router.delete("/tracks/{track_id}/instruments/{instrument_id}",
               status_code=status.HTTP_204_NO_CONTENT, responses={...})
def remove_instrument(track_id: Annotated[int, Path(ge=1)],
                      instrument_id: Annotated[int, Path(ge=1)],
                      db: Db, current_user: CurrentUser):
    track_service.remove_instrument(db, current_user, track_id, instrument_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

문법 풀이

식	뜻
`response_model=list[TrackInstrumentResponse]`	목록 경로는 배열을 돌려줍니다. 감싸는 객체가 없습니다
`[TrackInstrumentResponse.model_validate(i) for i in instruments]`	ORM 목록을 응답 목록으로 한 줄에 바꿉니다
경로 값 두 개(`track_id`, `instrument_id`)	둘 다 `Path(ge=1)` 입니다. 주소에 있는 번호가 함수 인자로 들어옵니다
`req: TrackInstrumentCreateRequest`	본문(JSON)을 검사해 넘깁니다. 검사 실패는 422 입니다
서비스에 `track_id` 와 `instrument_id` 를 함께 넘김	악기가 정말 그 트랙의 것인지 서비스가 확인합니다. 아니면 404 입니다

4. 왜 이렇게

선택	이유
악기 번호를 칸 번호 대신 씀	주소의 `{instrument_id}` 는 행 번호(기본 키)입니다. 칸 번호는 바뀔 수 있어 주소로 쓰면 헷갈립니다
붙이기만 201	새 행이 생기는 경로만 201 입니다. 고치기와 떼기는 각각 200, 204 입니다
파라미터 전체를 한 번에	노브 하나마다 요청을 보내면 소리가 끊깁니다. 화면이 모아 보내고 `params` 를 통째로 바꿉니다 (FR-262)

선택	이유
`exclude_unset` 로 부분 수정	볼륨만 고칠 때 나머지 칸이 0 으로 덮이지 않습니다 (238 과 같은 방식)

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 붙이기 201과 응답 칸 이름, `params` 저장, 기본값(`is_enabled` = true)
- 없는 플러그인 404, 이펙트 플러그인을 넣으면 400
- 같은 칸 두 번이면 400 (`test_slot_conflict_is_400`), 자기 자리로의 수정은 200
- 고치기 응답(볼륨·켜기), 떼기 204와 목록 비기

Backend(FastAPI/Python) 240 — 이펙트 API (목록 · 붙이기 · 고치기 · 떼기)

항목	내용
파일	`studionx/backend/app/api/v1/tracks.py` (이펙트 경로)
작업	S1-09 기본 악기·이펙트: 트랙 안 이펙트 칸 경로 네 개 (PRD 02 FR-245, FR-263, FR-264)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	239 악기 API · 241 연결선 API

1. 하는 일

트랙의 이펙트 체인을 다루는 경로 네 개를 엽니다.

경로	하는 일	상태 코드
`GET /tracks/{track_id}/plugins`	이펙트 목록 (칸 번호 순 = 체인 순서)	200, 401, 404
`POST /tracks/{track_id}/plugins`	이펙트 붙이기	201, 400, 401, 403, 404
`PATCH /tracks/{track_id}/plugins/{track_plugin_id}`	순서·카드 자리·파라미터·바이패스 고치기	200, 400, 401, 403, 404
`DELETE /tracks/{track_id}/plugins/{track_plugin_id}`	이펙트 떼기	204, 401, 403, 404

주소 이름이 `/plugins` 인 이유는 화면이 부르는 이름(이펙트 체인)을 그대로 쓰기 위해서입니다. 표 이름(`track_plugins`)과 경로 번호 이름(`track_plugin_id`)은 서비스·DB 와 맞춰 두었습니다.

2. 언제 쓰나

쓰는 곳	
화면의 [Effects] 칸	체인 카드를 그릴 때 `GET`, 새 이펙트를 붙일 때 `POST`
카드 끌어 순서 바꾸기	옮긴 카드의 새 칸 번호를 `PATCH` 로 보냅니다 (FR-263)
바이패스 단추	`PATCH` 로 `is_bypassed` 만 보냅니다
갈래(패처) 화면	카드를 옮긴 자리를 `PATCH` 로 `node_x`·`node_y` 에 저장합니다 (FR-269)

3. 코드

목록과 붙이기

```
@router.get("/tracks/{track_id}/plugins", response_model=list[TrackPluginResponse],
            responses={...})
def list_effects(track_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    effects = track_service.list_effects(db, current_user, track_id)
    return [TrackPluginResponse.model_validate(p) for p in effects]
```

```
@router.post("/tracks/{track_id}/plugins", response_model=TrackPluginResponse,
             status_code=status.HTTP_201_CREATED, responses=WRITE_RESPONSES)
def add_effect(track_id: Annotated[int, Path(ge=1)], req: TrackPluginCreateRequest,
               db: Db, current_user: CurrentUser):
    tp = track_service.add_effect(
        db, current_user, track_id,
        plugin_id=req.plugin_id, slot_index=req.slot_index, node_x=req.node_x, node_y=req.node_y,
        params=req.params, is_bypassed=req.is_bypassed,
    )
    return TrackPluginResponse.model_validate(tp)
```

고치기와 떼기

```
@router.patch("/tracks/{track_id}/plugins/{track_plugin_id}",
              response_model=TrackPluginResponse, responses=WRITE_RESPONSES)
def update_effect(track_id: Annotated[int, Path(ge=1)],
                  track_plugin_id: Annotated[int, Path(ge=1)],
                  req: TrackPluginUpdateRequest, db: Db, current_user: CurrentUser):
    data = req.model_dump(exclude_unset=True)
    tp = track_service.update_effect(db, current_user, track_id, track_plugin_id, **data)
    return TrackPluginResponse.model_validate(tp)


@router.delete("/tracks/{track_id}/plugins/{track_plugin_id}",
               status_code=status.HTTP_204_NO_CONTENT, responses={...})
def remove_effect(track_id: Annotated[int, Path(ge=1)],
                  track_plugin_id: Annotated[int, Path(ge=1)],
                  db: Db, current_user: CurrentUser):
    track_service.remove_effect(db, current_user, track_id, track_plugin_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

문법 풀이

식	뜻
`{track_plugin_id}`	카드 한 장의 행 번호입니다. 235 의 `_require_effect` 가 트랙과 맞는지 확인합니다
`req: TrackPluginCreateRequest`	`slot_index` 가 반드시 있어야 합니다(기본값 없음). 없으면 422 입니다
`exclude_unset=True`	순서만 바꿀 때 파라미터가 빈 사전으로 덮이지 않게 합니다
응답을 카드 한 장으로	붙이기·고치기는 바뀐 카드를 그대로 돌려줍니다. 화면이 카드만 다시 그리면 됩니다
목록 응답이 배열	연결선·악기 목록과 같은 모양입니다. 개수는 화면이 `length` 로 셉니다

4. 왜 이렇게

선택	이유
순서 바꾸기를 `PATCH` 로	옮긴 카드 한 장만 보내면 됩니다. 목록 전체를 다시 보내는 방식은 동시 편집에서 서로 덮어씁니다
이미 찬 칸이면 400	같은 칸 번호가 둘이면 소리 순서를 정할 수 없습니다. 서비스가 DB UNIQUE 전에 확인합니다 (235)

선택	이유
카드 자리를 같은 경로에서 수정	자리와 순서는 별개입니다. 자리만 바꾸면 소리 순서는 그대로입니다 (FR-269)
이펙트 플러그인만 허용	악기를 체인에 넣으면 신호 흐름이 끊깁니다. 400 으로 알려 줍니다 (FR-245)

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 붙이기 201과 응답 칸 이름, `node_x` 저장, 기본값(`is_bypassed` = false)
- 악기 플러그인을 넣으면 400, 없는 플러그인 404
- 목록 개수, 바이패스 수정, 칸 번호 2로 옮기기, 남이 쓰는 칸으로 옮기면 400
- 떼기 204

Backend(FastAPI/Python) 241 — 연결선 API (목록 · 긋기 · 지우기)

항목	내용
파일	`studionx/backend/app/api/v1/tracks.py` (연결선 경로)
작업	S1-09 기본 악기·이펙트: 갈래 연결선 경로 세 개 (PRD 02 FR-265~269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	240 이펙트 API · 242 시드 SQL

1. 하는 일

갈래(패처) 연결선을 다루는 경로 세 개를 엽니다.

경로	하는 일	상태 코드
`GET /tracks/{track_id}/links`	연결선 목록 (번호 순)	200, 401, 404
`POST /tracks/{track_id}/links`	연결선 긋기	201, 400, 401, 403, 404
`DELETE /tracks/{track_id}/links/{link_id}`	연결선 지우기	204, 401, 403, 404

연결선을 고치는 경로는 아직 없습니다. 선을 옮기는 화면이 필요하면 지우고 다시 긋거나, 준비해 둔 `TrackPluginLinkUpdateRequest`(230)로 경로를 하나 더 열면 됩니다.

2. 언제 쓰나

쓰는 곳	
갈래(패처) 보드 열기	`GET` 으로 저장된 선을 그립니다 (FR-267)
카드 이어 붙이기	카드 출력에서 다른 카드 입력으로 끌어 놓으면 `POST` (FR-265)
선 지우기	선을 고르고 `DELETE`

3. 코드

목록과 긋기

```
@router.get("/tracks/{track_id}/links", response_model=list[TrackPluginLinkResponse],
            responses={...})
def list_links(track_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    links = track_service.list_links(db, current_user, track_id)
    return [TrackPluginLinkResponse.model_validate(lnk) for lnk in links]


@router.post("/tracks/{track_id}/links", response_model=TrackPluginLinkResponse,
             status_code=status.HTTP_201_CREATED, responses=WRITE_RESPONSES)
def add_link(track_id: Annotated[int, Path(ge=1)], req: TrackPluginLinkCreateRequest,
             db: Db, current_user: CurrentUser):
```

```
    link = track_service.add_link(
        db, current_user, track_id,
        from_track_plugin_id=req.from_track_plugin_id, from_port=req.from_port,
        to_track_plugin_id=req.to_track_plugin_id, to_port=req.to_port, gain_db=req.gain_db,
    )
    return TrackPluginLinkResponse.model_validate(link)
```

지우기

```
@router.delete("/tracks/{track_id}/links/{link_id}",
               status_code=status.HTTP_204_NO_CONTENT, responses={...})
def remove_link(track_id: Annotated[int, Path(ge=1)], link_id: Annotated[int, Path(ge=1)],
                db: Db, current_user: CurrentUser):
    track_service.remove_link(db, current_user, track_id, link_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

문법 풀이

식	뜻
`from_track_plugin_id=req.from_track_plugin_id`	요청 칸 이름을 그대로 서비스 인자로 넘깁니다. `from` 은 파이썬 예약어라 접두어를 붙였습니다
`from_port`·`to_port`	카드의 어느 입·출력에 꽂혔는지 나타냅니다. 0 부터 셉니다
`gain_db`	선마다 다른 세기입니다. 기본 0.00 dB
`{link_id}`	선 한 줄의 행 번호입니다. 트랙과 맞지 않으면 404

4. 왜 이렇게

선택	이유
표 이름(`link`)을 주소에 그대로	화면 코드와 DB 가 같은 이름을 씁니다. `connection` 처럼 다른 말을 쓰면 트랙 사이 연결(`track_connections`)과 헷갈립니다
긋기만 201	새 선이 생기는 경로입니다. 지우기는 204
선 수정 경로를 안 엶	지금 화면에는 선을 끌어 옮기는 기능이 없습니다. 안 쓰는 경로를 미리 열면 시험도 문서도 늘어납니다
목록에 번호 순 정렬	선이 그려지는 순서가 매번 같아 화면이 흔들리지 않습니다

5. 시험

`tests/test_track_routes.py` 의 `test_patcher_links_crud` 가 트랙 입력(빈 `from`)에서 이펙트 A 로 가는 선과 이펙트 A 에서 이펙트 B 로 가는 선을 만들고, 목록 개수·`gain_db` 저장·지우기를 확인합니다. 다른 트랙의 이펙트를 가리키면 404 입니다.

Backend(FastAPI/Python) 242 — 시드 SQL (기본 악기·이펙트 일곱 종)

항목	내용
파일	`studionx/db/migrations/2026-10-18_plugins_seed.sql` (같은 내용이 `studionx/db/02_phase2_sequencer.sql` 에도 들어 있습니다)
작업	S1-09 기본 악기·이펙트: 플러그인 정의 일곱 종을 넣습니다 (PRD 02 FR-241, FR-244, FR-246)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	241 연결선 API · (다음) 78 개요

1. 하는 일

`plugins` 표에 기본 플러그인 일곱 종을 넣습니다. 악기 둘과 이펙트 다섯입니다. 각 줄에는 화면이 노브와 선택 상자를 그리는 데 필요한 `param_schema`(JSON)가 함께 들어갑니다.

code	이름	종류	항목 수	나눔(group)
`synth_sub`	Subtractive Synth	INSTRUMENT	17	Osc 1, Osc 2, Filter, Amp Env, LFO
`sampler`	Sampler	INSTRUMENT	8	Sample, Amp Env
`eq3`	EQ	EFFECT	7	Low, Mid, High
`compressor`	Compressor	EFFECT	6	Main
`reverb`	Reverb	EFFECT	4	Main
`gain`	Gain / Utility	EFFECT	3	Main
`limiter`	Limiter	EFFECT	2	Main

모두 `engine` = `WEBAUDIO`, `version` = `1.0.0`, `is_active` = 1 입니다. 항목 47개 가운데 41개는 종류를 적지 않아 `knob`(돌리는 노브)이 되고, 선택 상자 4개와 켜기/끄기 2개만 종류를 적었습니다.

2. 언제 쓰나

쓰는 곳	
처음 DB 를 만들 때	`studionx/db/02_phase2_sequencer.sql` 에 같은 `INSERT` 가 들어 있어 따로 실행하지 않습니다
이미 쓰던 DB	이 파일을 실행해 일곱 종을 넣거나 갱신합니다 (예: 정의를 고쳤을 때)
시험	`tests/test_plugin_routes.py` 가 `synth_sub`·`eq3` 를 골라 읽습니다
화면	`GET /plugins` 가 이 표를 읽어 악기/이펙트 고르기 목록을 만듭니다

3. 코드

```
INSERT INTO plugins (code, name, plugin_type, engine, version, param_schema, card_asset, is_active)
VALUES
('synth_sub', 'Subtractive Synth', 'INSTRUMENT', 'WEBAUDIO', '1.0.0',
```

```
  '[{"key":"osc1_wave","name":"Wave","type":"select","group":"Osc 1","default":"saw",
     "options":[{"value":"sine","label":"Sine"}, ...]},
    {"key":"cutoff","name":"Cutoff","group":"Filter","min":20,"max":20000,"default":1200,
     "unit":"Hz","step":1,"curve":"log","primary":true}, ...]',
 NULL, 1),
('sampler', 'Sampler', 'INSTRUMENT', 'WEBAUDIO', '1.0.0', '[ ... ]', NULL, 1),
('eq3', 'EQ', 'EFFECT', 'WEBAUDIO', '1.0.0', '[ ... ]', NULL, 1),
('compressor', 'Compressor', 'EFFECT', 'WEBAUDIO', '1.0.0', '[ ... ]', NULL, 1),
('reverb', 'Reverb', 'EFFECT', 'WEBAUDIO', '1.0.0', '[ ... ]', NULL, 1),
('gain', 'Gain / Utility', 'EFFECT', 'WEBAUDIO', '1.0.0', '[ ... ]', NULL, 1),
('limiter', 'Limiter', 'EFFECT', 'WEBAUDIO', '1.0.0', '[ ... ]', NULL, 1)
ON DUPLICATE KEY UPDATE
  name = VALUES(name),
  plugin_type = VALUES(plugin_type),
  engine = VALUES(engine),
  version = VALUES(version),
  param_schema = VALUES(param_schema),
  card_asset = VALUES(card_asset),
  is_active = VALUES(is_active);
```

문법 풀이

식	뜻
`code`	플러그인을 가리키는 짧은 이름입니다. `UNIQUE` 라 같은 `code` 로 다시 넣으면 갱신됩니다
`param_schema`	화면이 노브를 그리는 설계도입니다. `key`(저장 이름), `name`(보이는 이름), `min`·`max`·`default`·`step`, `unit`, `type`, `group`, `curve`, `primary`, `options` 를 담습니다
`"type":"select"`	고르는 상자입니다. `options` 의 `value`·`label` 을 씁니다
`"type":"toggle"`	켜기/끄기 단추입니다. 값을 적지 않으면 `knob` 입니다 (229)
`"curve":"log"`	소리에서 자주 쓰는 로그 눈금입니다. `Cutoff` 처럼 넓은 범위에 씁니다
`"primary":true`	카드 앞면에 크게 보여 줄 주요 노브입니다
`card_asset` = NULL	카드 그림은 화면(CSS)이 그립니다. 그림 파일을 쓰는 플러그인이 생기면 여기에 경로를 넣습니다
`ON DUPLICATE KEY UPDATE`	같은 `code` 가 있으면 값을 새로 씁니다. 여러 번 실행해도 같은 결과입니다

4. 왜 이렇게

선택	이유
정의를 코드가 아니라 시드로	플러그인은 소리 엔진(`WEBAUDIO`)이 아는 것만 넣을 수 있습니다. 화면에서 만들면 없는 엔진 이름이 들어갑니다
`code` 를 키로 갱신	정의를 고칠 때 지우고 다시 넣으면, 그 플러그인을 쓰던 트랙의 악기·이펙트가 함께 지워집니다. 갱신이 안전합니다
일곱 종으로 시작	신시사이저, 샘플러, EQ, 컴프레서, 리버브, 게인, 리미터면 S1-09 화면 검토에 충분합니다. 더 필요한 종류는 나중에 같은 방식으로 넣습니다
`synth_sub` 만 17개	소리를 만드는 악기는 조절할 것이 많습니다. 이펙트는 3~7개로 짧게 두어 카드가 복잡해지지 않게 했습니다
항목 이름을 영어로	코드 이름(`cutoff`)과 화면 이름(`Cutoff`)을 그대로 씁니다. 한글 이름은 화면 사전이 붙일 때 바꿉니다

5. 시험

- `tests/test_plugin_model.py` 가 `param_schema` 를 실제 MySQL 에 넣었다 다시 읽어 같은 사전이 되는지 확인합니다 (JSON 왕복).
- `tests/test_plugin_routes.py` 가 시드에서 `synth_sub·eq3` 를 고유 `code` 로 만들어 읽고, `param_schema` 가 비어 있지 않은지 확인합니다.
- 시드 파일을 두 번 실행해도 오류가 나지 않습니다(`ON DUPLICATE KEY UPDATE`).